EQuack: Eventually Consistent Access Control through Deterministic Replay

Aditya Sissodiya, Johan Kristiansson, Eric Chiquito and Ulf Bodin
Luleå University of Technology, Sweden

Abstract—Intermittently connected multi-stakeholder systems need authorization decisions that remain available offline yet converge deterministically after partitions heal. EQUACK is an eventually consistent access-control model that records data operations, credential grants and revocations, and trust evidence in a single signed causal event log. Replicas merge logs by set union and reconstruct state through deterministic replay with a deny-wins gate, so replicas that have received the same well-formed events compute the same authorized state, operations admitted under subsequently revoked credentials are excluded after reconciliation. In a Rust prototype, 100 valid topological orderings of a 10 000-operation log produced an identical state digest; all 500 post-revocation writes in a policy workload were suppressed after reconciliation; multi-issuer conflicts resolved deterministically; and replay reached 45 000 operations/s with 8–11× faster recovery from checkpoints.

Index Terms—Access control, revocation, verifiable credentials, CRDTs, deterministic replay, audit, offline systems, industrial IoT, data spaces.

1 INTRODUCTION

Access control in multi-stakeholder environments faces an unresolved tension; devices and services must continue operating when disconnected, yet security-critical decisions, especially credential revocation, must eventually be enforced consistently across all parties. The problem is sharpest in industrial Internet of Things (IoT) [1], [2], healthcare [3], and cross-organizational data sharing [4], [5], where equipment changes hands between manufacturers, operators, and service providers throughout its lifecycle (e.g., a compressor sold by vendor A, operated by mining company B, and serviced by maintenance contractor C, each holding distinct permissions over its control parameters).

Federated research data sharing [3], [4], disaster-response coordination, and industrial control share an enforcement gap; independent authorities issue and revoke credentials, devices operate offline, and permissions evolve while nodes are disconnected. Authorization in these settings is an *offline-first* problem; nodes must decide locally, yet the system must later reconcile those decisions into a consistent, auditable state.

Consider a factory running hundreds of programmable controllers on a restricted network. Maintenance engineers from different vendors receive temporary access to specific machines, but controllers may operate for days without reliable connectivity [6], [7]. An engineer updates a configuration parameter while offline using a credential from their employer; hours later, that credential is revoked after a safety incident. When logs from different controllers are synchronized, the system must deterministically agree on which parameter changes are retained in the converged digital state, and produce an audit trail linking each change to the actor, the permission, and the time. EQUACK governs which configuration changes are retained in the reconciled state; it does not govern whether a controller *immediately acts* on a parameter change before reconciliation. Applications requiring staged execution of safety-critical commands must implement hold-and-confirm logic at the application layer (Section 5).

Among the approaches surveyed in Section 2.4, none provides deterministic post-partition reconciliation of fine-grained authorization decisions. Centralized systems require online coordination; token and capability schemes verify offline but cannot reconcile conflicting authorization state; Conflict-free Replicated Data Types (CRDTs) converge data but not authorization semantics; and ledger-based designs require consensus incompatible with disconnected operation (Section 2.4).

EQUACK targets *authorization-rate* workloads: access-control decisions issued at the rate of human or administrative actions (operations per minute, not per millisecond). It is not designed for high-throughput data-plane enforcement such as per-packet filtering or real-time telemetry ingestion; Section 5 expands on this positioning.

Jacob and Hartenstein [8] formalize this setting as *Eventually Consistent Access Control* (ECAC); replicas autonomously enforce access control from local knowledge and reconcile after reconnection. Their model distinguishes *causal authorization* (immutable, based on predecessor history) from *topological authorization* (mutable, based on the total execution order), and provides formal safety and liveness properties, including app state confluence under non-monotonic revocations, but does not specify how credentials are represented, how trust evidence propagates offline, how conflicting grants and revokes from independent issuers are resolved, or how fine-grained attribute-based policies are evaluated over reconciled history. They provide no implementation or experimental evaluation. Kleppmann et al. [9] independently identify access control, particularly revocation, as an unsolved problem for local-first CRDT-based software.

This paper presents EQUACK, an instantiation of the ECAC paradigm that closes these mechanism-level gaps. EQUACK records data operations, credential grants and revocations, and trust evidence in a single signed causal event log. Replicas merge logs by set union and reconstruct authorized state through deterministic replay with a *deny-wins* gate, a conflict-resolution rule in which any revocation overrides concurrent permits. Authorization is expressed through Verifiable Credential (VC)-based time-bounded epochs evaluated by a Cedar-inspired policy engine supporting Attribute-Based Access Control (ABAC) and Role-Based Access Control (RBAC) semantics, so that fine-grained, scoped, attribute-conditional permissions are enforceable offline and reconcilable after partition.

The novelty is not the individual ingredients; CRDTs, VCs, policy engines, but the replay mechanism that composes them into a deterministic authorization function over reconciled history, providing properties (R2, R3) that—among the systems surveyed in Sections 2.4–2.5—we are not aware of being achieved simultaneously with fine-grained policy expressiveness.

This paper makes the following contributions.

1) *ECAC instantiation with fine-grained policy support* (Sections 2–3). We instantiate the ECAC paradigm with VC-based authorization epochs, in-band trust propagation, and Cedar-inspired policy evaluation; mechanisms that ECAC defines abstractly but does not specify.
2) *Deterministic deny-wins reconciliation* (Section 3). We define a causal Directed Acyclic Graph (DAG) linearization and replay gate that produces the same authorized state on every replica that has received the same events. Operations admitted under subsequently revoked credentials are excluded after reconciliation.
3) *Prototype and evaluation* (Sections 3–4). *equack-core* is evaluated for convergence, revocation enforcement, multi-authority conflict resolution, audit integrity, and replay cost. The experiments confirm identical state digests across 100 topological orderings of a 10 000-operation log, suppression of all post-revocation writes, deterministic conflict resolution, tamper detection, and replay throughput up to 45 000 ops/s with 8–11× checkpoint speedup.

Section 2 formalizes the problem and derives requirements. Section 3 presents the EQUACK model, its Rust prototype implementation, and formal claims. Section 4 evaluates it, Section 5 discusses trade-offs and positioning, and Section 6 concludes.

2 Problem Analysis

The offline multi-stakeholder authorization problem arises when systems must (i) continue operating during network partitions, (ii) support multiple independent authorities that can grant and revoke permissions, and (iii) eventually converge to a consistent, auditable authorization state. This section defines the problem context, presents conceptual requirements derived from motivating use cases, and evaluates existing approaches against those requirements.

The requirements and comparisons that follow assume an offline-first, multi-authority setting in which replicas may be disconnected for extended periods, multiple independent organizations issue and revoke credentials, and a computationally bounded adversary may delay, reorder, or partition messages. Section 2.2 gives the complete system and threat model; the brief characterization here is sufficient to interpret R1–R6 and the subsequent gap analysis. This setting explains why common fixes fail; online introspection endpoints violate offline-first operation; global consensus violates availability during partition; single-authority trust violates multi-stakeholder governance.

2.1 The Offline Enforcement Problem

Consider a set of replicas $R = \{r_1, \ldots, r_n\}$ operated by multiple stakeholders, maintaining shared resources under access control. Each replica may be offline for extended periods, during which it must make local authorization decisions. When replicas reconnect, they must reconcile their local states into a globally consistent outcome.

The core difficulty is not issuing permissions, standard Public Key Infrastructure (PKI) and token mechanisms suffice for that, but ensuring that all replicas eventually agree on *which operations were authorized at the time they occurred*, despite:

- *Partition-time divergence*: Different replicas may observe different subsets of grants, revokes, and data operations during a partition.
- *Out-of-order delivery*: A revocation may arrive at a replica after operations it should have invalidated.
- *Multi-authority conflicts*: Independent issuers may make conflicting decisions (e.g., one grants while another revokes) that must be resolved deterministically.

Figure 1 illustrates these three challenges and the convergence pipeline that resolves them; after reconnection, replicas merge their event sets, establish a deterministic total order, and replay authorization decisions under a deny-wins policy to reach identical verdicts.

Jacob and Hartenstein [8] formalize this setting as ECAC (Section 1). Their abstract algorithm proves safety and liveness properties, including app state confluence under non-monotonic revocations, but operates at the level of Matrix's power-level-based authorization (LeABAC) rather than credential-based epochs with policy-engine evaluation. EQUACK instantiates their paradigm with concrete mechanisms.

2.2 System and Threat Model

We consider a set of replicas operated by multiple stakeholders that jointly maintain a replicated log of operations over shared resources. Replicas are deployed in environments with intermittent connectivity (e.g., edge and industrial deployments) and may be offline for extended periods. Each replica locally stores all received operations and materializes application state by deterministic replay.

A *subject* (principal) is an actor that initiates operations (users, services, devices). One or more independent *issuers* create and revoke time-bounded credentials for subjects. Each replica hosts a policy enforcement point (PEP) that admits subject requests only when locally authorized, then emits a signed log event carrying the subject identity and

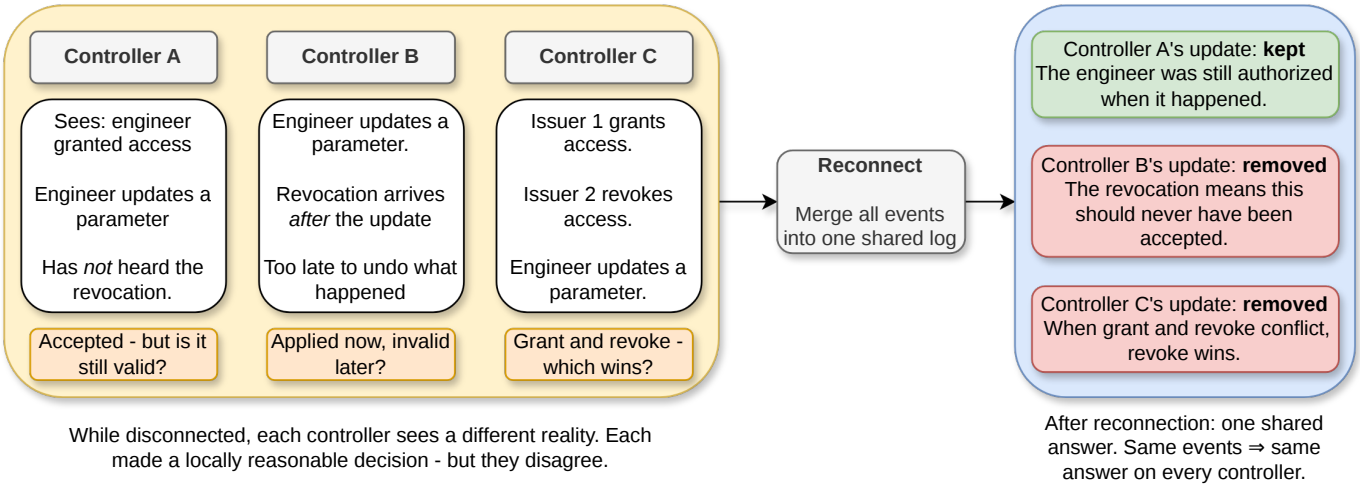

Fig. 1. The offline enforcement problem. While disconnected, each controller makes authorization decisions from an incomplete view of grants and revocations. After reconnection, the system deterministically agrees on which operations remain authorized, despite late-arriving revocations and conflicting decisions from independent authorities. The hard part is not granting or revoking access. The hard part is making every device agree afterward on what was authorized.

authorization evidence. The event timestamp (HLC) and signature are produced by the replica/PEP, not the subject, preventing forging or backdating under our threat model. All state changes are represented as signed operations in a causal DAG, disseminated via gossip/anti-entropy over an asynchronous network where messages may be delayed, reordered, duplicated, or temporarily dropped.

EQUACK targets security failures arising from *offline enforcement* and *multi-authority governance*. We assume an adversary may (i) compromise subject credentials and attempt unauthorized operations, (ii) control the network to delay, reorder, or partition replicas, and (iii) attempt to tamper with stored history. The adversary cannot forge replica/PEP signing keys or choose HLC timestamps, and is computationally bounded.

The following assumptions scope the guarantees:

1) *Offline-first operation.* Replicas may be disconnected for hours to days; solutions requiring continuous connectivity are excluded.
2) *Multi-authority governance.* Multiple independent organizations issue and revoke credentials; no single root authority controls all permissions.
3) *Honest-but-buggy administrators.* Issuers follow protocols but may make errors; EQUACK provides safety-by-default semantics.
4) *Eventual delivery.* Every well-formed operation is eventually delivered to every correct replica, though delivery order may be adversarially influenced.
5) *Bounded partition duration.* Offline periods are finite; replicas reconcile on reconnection.
6) *No physical key compromise.* Long-term private keys are not exfiltrated via physical attacks; compromise is modeled at the software/account level.
7) *Bounded clock skew.* Each replica's physical clock is within a known bound ϵ of real time. The HLC's physical component inherits this bound; the logical counter and node identifier break ties deterministically regardless of skew. Credential validity intervals $[\mathrm{nb}, \mathrm{na})$ should be configured with margins exceeding ϵ to avoid boundary ambiguity. Replicas with unbounded clock drift may make incorrect local authorization decisions that are corrected on replay only when the replayed HLC values remain consistent with the true causal order.

EQUACK does not address fully Byzantine collusion, side-channel leakage, cryptographic primitive failures, or metadata privacy; these require additional mechanisms.

Scope of guarantees: EQUACK guarantees convergence and revocation safety for the *digital authorization state*: which operations are retained or excluded in the replicated log after reconciliation. It does *not* prevent a controller from acting on a locally accepted operation before that operation is confirmed or invalidated by replay. In safety-critical deployments where physical actions (actuator commands, firmware updates, safety-parameter writes applied to running equipment) must not execute under stale authorization, the application layer must defer execution until sync confirms the operation's authorization status. EQUACK provides the mechanism for determining that status; it does not prescribe when the application acts on it. The hold-and-confirm pattern described in Section 5 is one such application-layer policy that uses EQUACK's replay verdicts as input.

2.3 Conceptual Requirements

From the system context and motivating use cases (Section 1), we derive six conceptual requirements for offline multi-stakeholder access control. These serve as criteria for comparing existing paradigms and guiding EQUACK's design.

1) ***R1—Offline enforceability.*** Authorization decisions must be enforceable without contacting a central Policy Decision Point (PDP) or ledger; disconnected nodes operate using locally verifiable evidence.
2) ***R2—Deterministic post-partition convergence.*** Given the same set of events, all parties must compute the

same final authorization state regardless of message arrival order.

3) ***R3—Revocation safety under partition.*** Revocations must deterministically invalidate effects in the replicated state after reconciliation, even if the revocation arrives after the operations it should have blocked. Revocation applies to the digital state; irreversible physical-world actions require compensating controls at the application layer.
4) ***R4—Tamper-evident accountability.*** The history of grants, revokes, and authorization decisions must form an append-only, verifiable trace.
5) ***R5—Decentralized multi-authority issuance.*** Independent issuers must grant and revoke permissions without coordination; verifiers check authenticity offline using locally available evidence.
6) ***R6—Confidentiality with post-revocation protection.*** Unauthorized parties should not read protected content; after revocation-triggered key rotation, revoked principals cannot decrypt data created after their access ended.

Explicit non-requirement. We do not address data-plane throughput requirements: EQUACK assumes that authorization decisions are issued at rates compatible with human or administrative workflows (Section 1), not at per-packet or telemetry-ingest rates.

2.4 Gap Analysis

Table 1 evaluates five dominant approaches against R1–R6. Centralized Identity and Access Management (IAM) couples enforcement to online coordination, failing R1 and R3 under partition [8]. Token-based schemes enable offline checks but lack deterministic reconciliation and timely revocation [10]. CRDTs converge data payloads; they do not define authorization semantics or revocation rollback [11], [12]. Blockchains offer total ordering and auditability at the cost of online consensus, which is incompatible with offline enforcement [13]–[20]. Capability-based systems [21]–[23] verify delegated authority offline but cannot deterministically converge conflicting authorization state or retroactively revoke. Among the paradigms surveyed here, none simultaneously satisfies R1–R6.

Table 1 identifies which *properties* each paradigm satisfies. The deeper gap is mechanism-level; among the approaches surveyed, none provides (i) deterministic reconciliation of authorization history after partition, (ii) deny-wins revocation semantics that retroactively exclude operations admitted under stale credentials, (iii) fine-grained policy evaluation (ABAC/RBAC) over the reconciled history, (iv) in-band propagation of trust and credential-status evidence, or (v) auditable linkage between credential state and data-state replay. ECAC [8] provides the formal framework for (i) but not the concrete mechanisms for (ii)–(v). EQUACK addresses these mechanism-level gaps. This claim is based on the paradigms tabulated in Table 1 and the specific systems compared in Section 2.5; we do not claim exhaustive coverage of all distributed authorization systems.

2.5 Mechanism-Level Comparison

To clarify EQUACK's positioning, Table 2 compares it against the closest design patterns along five axes; ordering model, revocation semantics, trust distribution, audit model, and offline availability.

EQUACK fills the gap between these approaches, CRDT replication provides data convergence but not authorization convergence [11], [12]; ledgers provide total order but require consensus; token/VC designs support offline checks but lack deterministic reconciliation [10]. EQUACK's causal DAG linearization plus deny-wins gate makes authorization a reproducible function of the reconciled event set without online consensus.

Capability-based systems—Macaroons [21], UCAN [22], Biscuit [24], WAVE [23], and the Willow Protocol [25]—enable offline verification of delegated authority (R1, R5) but do not address deterministic convergence of conflicting authorization state (R2) or retroactive invalidation of effects authorized under stale evidence (R3). Revocation relies on expiry or propagation, facing the same freshness problem as VC status lists. EQUACK's contribution is orthogonal; it provides the replay mechanism that deterministically reconciles authorization state after partition healing.

Zanzibar [26] and its open-source derivatives (e.g., OpenFGA, SpiceDB) provide relationship-based authorization backed by strongly consistent stores, violating R1 by construction. Policy engines such as Open Policy Agent (OPA) and Cedar [27] support offline policy *evaluation* but not offline policy *modification* with deterministic convergence. EQUACK sits between these extremes; offline operation with deterministic post-partition convergence, without global consensus.

Matrix's state resolution (v2) [28] is structurally similar; a DAG of events with deterministic state resolution across federated servers, but uses power-level-based authorization rather than CRDT formalism, deny-wins semantics, or VC-based epochs. Jacob and Hartenstein [8] formalize Matrix's approach as an ECAC instance; EQUACK builds on this foundation targeting industrial multi-stakeholder settings. EQUACK advances beyond the ECAC abstraction by specifying the credential format (W3C VCs), the conflict resolution semantics (deny-wins epochs), the policy evaluation engine (Cedar ABAC), and the trust propagation mechanism (in-band TRUST events).

The p2panda toolkit [29] is the closest existing system. Its `p2panda-auth` crate [30] implements a convergent access-control CRDT with a "strong removal" resolver over a causal DAG, and its encryption layer [31] uses Decentralized Continuous Group Key Agreement (DCGKA)-based group keys with forward secrecy via the Signal Double Ratchet. Three differences position EQUACK distinctly; (1) EQUACK enforces deny-wins replay at the protocol level, the Gate function (Section 3.6) is mandatory and the materialized state always reflects revocations (Theorem 3), whereas p2panda treats retroactive data invalidation as an application-layer choice; (2) EQUACK models authorization as VC-based epochs evaluated by a Cedar policy layer with multi-issuer trust propagated in-band, whereas p2panda uses group membership with four permission levels and no policy engine; (3) EQUACK provides an explicit hash-chained audit trail (Theorem 4), whereas p2panda's tamper evidence is implicit in the signed DAG. p2panda's encryption layer is more mature than EQUACK's experimental per-tag scheme.

TABLE 1
Gap analysis against conceptual requirements R1–R6. "Partial" indicates the requirement can be achieved only with additional coordination or significant residual risk.

Requirement	Centralized IAM	Token-based	CRDT-only	Blockchain	Capability-based
R1 Offline enforceability	No	Partial[a]	Yes	No	Yes
R2 Deterministic convergence	Yes	No	Yes	Yes	No
R3 Revocation safety	No	No	No	Partial[b]	Partial[c]
R4 Tamper-evident audit	No	No	No	Yes	Partial
R5 Multi-authority issuance	No	No	No	Yes	Yes
R6 Confidentiality	Partial[d]	Partial[e]	No	No	No

[a] Token-based: tokens can be verified offline, but revocation checking typically requires online status endpoints. [b] Blockchain: revocation can be committed to the ledger, but requires online consensus. [c] Capability-based: expiry-based revocation works offline; immediate revocation requires propagation. [d] Centralized: encryption at rest and in transit, but key management is centralized. [e] Token-based: tokens may carry encrypted claims, but no key rotation on revocation.

TABLE 2
Mechanism-level comparison of EQUACK to closest design alternatives. Each column represents a design paradigm; rows correspond to the five comparison axes.

Axis	CRDT-only replication	Token/VC offline checks	Ledger-based AC	Capability-based	EQUACK
Ordering model	Partial order / join-semilattice merge; no global history	No shared history; decisions are local + issuer freshness	Total order via consensus / finality	No shared history; delegation chains are acyclic but unordered	Causal DAG + deterministic linear extension (Alg. 1)
Revocation semantics	Not encoded into merge; revoked effects may persist post-merge	Depends on freshness/online status; partition window persists	Globally ordered but requires online consensus to commit	Expiry-based or propagation-dependent; no retroactive invalidation	Deny-wins epochs: REVOKE deterministically terminates prior/concurrent GRANT epochs; post-revoke effects skipped on reconciliation
Trust distribution	Out-of-band PKI; data convergence only	Out-of-band issuer infra (status endpoints, Online Certificate Status Protocol)	Trust anchored in ledger membership / consensus	Delegation chains from root capabilities; offline-verifiable	In-band TrustView: issuer keys + status evidence replicated as events; offline validation from pinned genesis
Audit model	Data history not necessarily reconstructible; depends on app-level logging	Depends on external logging; hard to reconstruct distributed decisions	Immutable ledger provides auditability (at consensus cost)	Token chain provides delegation provenance; no operation-level audit	Signed + hash-chained audit records; decisions reproducible from the log against pinned checkpoint digest
Offline availability	Yes for writes, but no safe authorization rollback	Partial (offline permit) but revocation timeliness requires online checks	No (must reach consensus to commit)	Yes for authorization checks; freshness-limited for revocation	Yes: local enforcement continues; digital-state safety repaired deterministically on merge

3 EQUACK: MODEL AND IMPLEMENTATION

EQUACK is an Eventually Consistent Access Control model designed for offline-first operation with model-level security and auditability guarantees, supported by prototype evidence. It combines four elements; (1) a causal event log with deterministic deny-wins replay, (2) verifiable credentials for time-bounded authorization epochs, (3) a hybrid Attribute-Based Access Control (ABAC)/Role-Based Access Control (RBAC) policy model with deny-overrides semantics, and (4) per-tag encryption with key rotation.

The prototype, *equack-core*[1], is a Rust library that uses Ed25519 for signatures, BLAKE3 for hashing, XChaCha20-Poly1305 for encryption, and SHA-256 for status list bitstrings. Operations are serialized in Concise Binary Object Representation (CBOR).

Each subsection first describes the conceptual mechanism, then shows how *equack-core* realizes it. The formal model (Section 3.6) is self-contained and does not depend on implementation choices. We state formal claims for convergence, safety, determinism, audit integrity, and leaver security (post-revocation confidentiality under key rotation). Figure 2 summarizes the four-phase lifecycle (Ingest, Validate, Replay, Query). The key idea is that replicas do not edit shared truth directly; they submit signed evidence to the

1. Source code: https://github.com/adityasissodiya/equack-core.

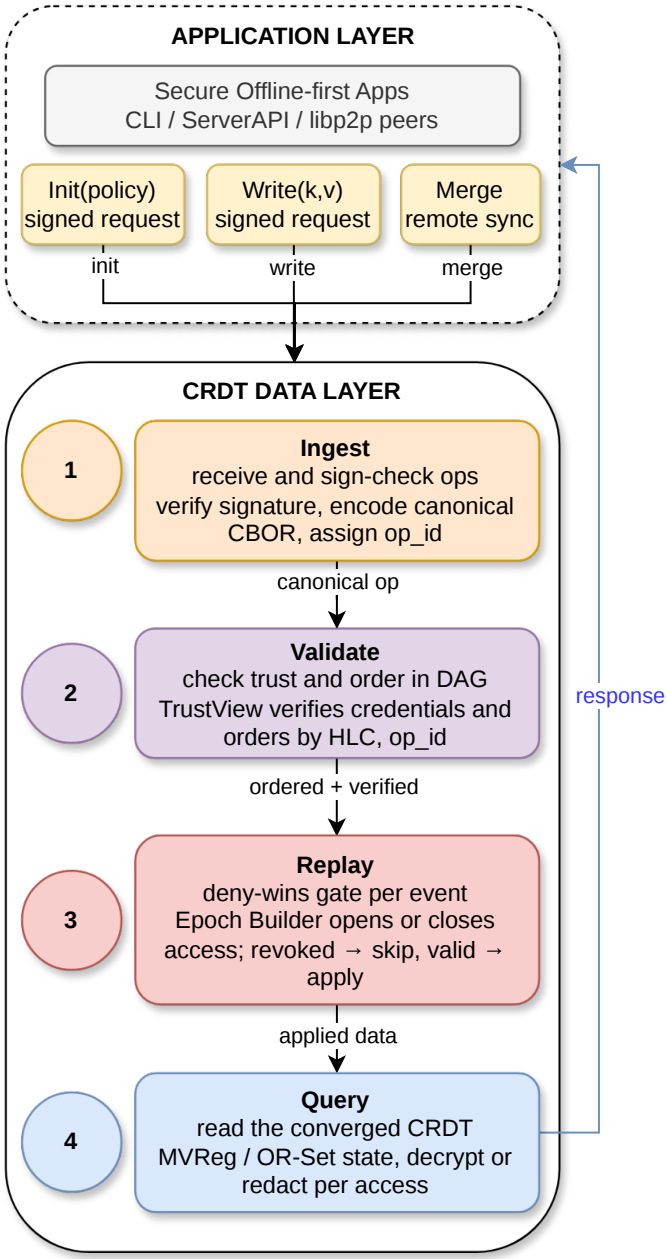

Fig. 2. EQUACK four-phase lifecycle. Events enter through Ingest, are validated by the TrustView and causally ordered in the DAG, pass through the deny-wins Epoch Builder + Gate, and converge in the CRDT State before reaching queries. Dashed arrows carry trust evidence and audit verdicts.

log first, and deterministic replay decides which evidence becomes state later.

3.1 Event Types and Causal DAG

EQUACK maintains an append-only event log structured as a causal DAG, where each event points to its causal predecessors. The DAG encodes happens-before relations, and system state is derived by deterministically sorting and replaying events. A fixed traversal order ensures every replica produces the same sequence.

Concurrent events, those without causal order, are resolved using a composite key; a Hybrid Logical Clock (HLC) value plus a unique operation ID. The HLC combines physical and logical time to preserve causality; when timestamps match, op IDs break ties deterministically. Replay thus becomes a pure function of the event set, guaranteeing identical state across replicas (R2).

A common point of confusion in offline access control is the overlapping role of causal graphs and Data CRDTs. In EQUACK, the pipeline follows a strict separation of concerns, the DAG and topological sort fix a deterministic, totally-ordered sequence of events to evaluate *authorization and policy validity*. Meanwhile, the CRDT state (e.g., MVReg, OR-Set) strictly handles *data payload convergence*. The policy engine evaluates the strict ordered DAG to build epochs and TrustViews; only authorized DATA events are then handed off to the CRDTs.

The event DAG acts as a convergent CRDT; merging logs is a set union, commutative, associative, idempotent. Operation structure and DAG mechanics are shown in Figure 3. Each operation carries a signed header with parent links, HLC timestamp, producer key, and typed payload. Replicas accept events opportunistically but only replay them once causal predecessors are present (activated/pending staging), ensuring every replica topologically sorts the same DAG prefix.

Replay follows a deny-wins rule, each event is applied only if the actor held the required permission at that moment; unauthorized events become no-ops. Operations are disseminated via gossip and background synchronization (anti-entropy) using libp2p; this affects liveness but not safety or determinism.

Implementation: The DAG is implemented in `core::dag`, which stores operations and computes the deterministic topological sort (`core::replay::topo_sort`). CRDT state (MVReg, OR-Set) is managed by `core::crdt`. Operations are serialized as deterministic CBOR with Ed25519 signatures computed over BLAKE3 hashes of the unsigned header (`core::crypto`). Gossip dissemination and anti-entropy synchronization use libp2p (`net::gossip`, `net::sync`).

3.2 Authorization Epochs and Credential Lifecycle

EQUACK manages permissions using verifiable, time-bounded credentials. It adopts the W3C VC format for signed authorization tokens asserting a principal's rights over a specific interval (validFrom–validUntil). Credentials are tamper-evident and self-verifying; replicas independently confirm authenticity and validity [32].

Revocations use standard VC status lists, cryptographically signed bitstrings indicating whether each credential ID is revoked. A valid credential defines an *authorization epoch*, the interval during which that credential grants a subject a specific permission; expiry or revocation ends it.

Trust evidence is carried in-band as TRUST events (issuer keys and status-list updates) and replayed into a local *TrustView*—the replica's materialized view of all issuer keys and credential-status evidence received so far; anchored by a pinned, cross-signed genesis checkpoint. The log forms a distributed PKI, an offline node can verify at any timestamp that a user held a valid, non-revoked credential without contacting a central authority.

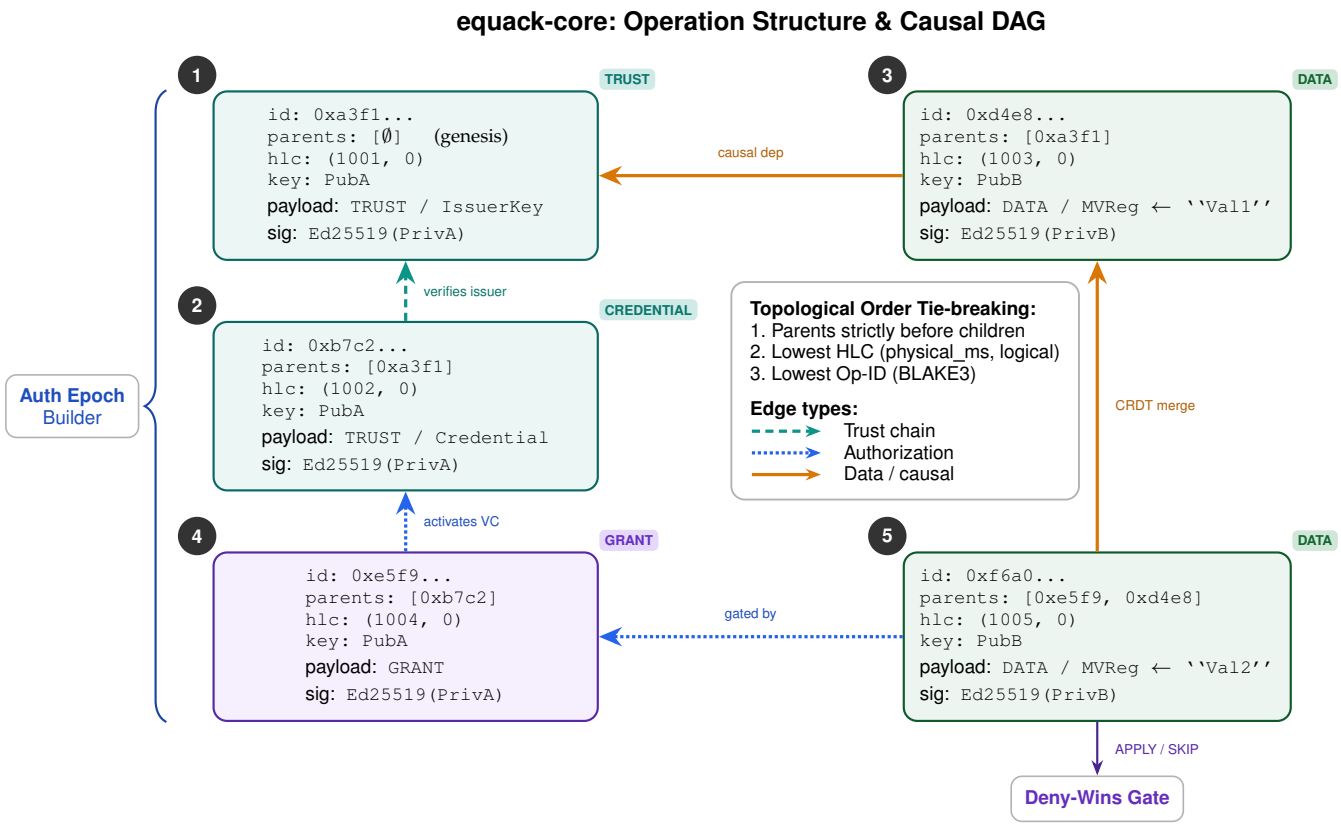

Fig. 3. Operation structure and causal DAG. Each node shows the OpHeader fields, typed payload, and Ed25519 signature. Edges encode three semantic relationships; trust-chain links (dashed teal) connect issuer keys to credentials, authorization links (dotted blue) connect credentials to grants and grants to gated data operations, and data/causal links (solid orange) encode CRDT merge points and ordering dependencies. Circled numbers indicate the deterministic replay order from Algorithm 1.

Genesis checkpoint governance: The pinned genesis checkpoint anchors the trust chain: it contains the initial set of issuer public keys and is intended to be cross-signed by all founding stakeholders. In the prototype, the genesis is created by a single administrative command and distributed out-of-band; the code path lives in `core::trustview` and `core::trust` and treats the genesis as the first entry in the replica's `TrustView`. A production deployment would require a multi-party ceremony in which each stakeholder independently verifies the genesis content before signing. Adding or removing stakeholders after genesis requires a signed `GenesisUpdate` event referencing the previous genesis digest; this mechanism is specified at the event-schema level but is not yet implemented in the prototype. The governance protocol for genesis updates (quorum requirements, key rotation for departing stakeholders, dispute resolution) is an open design problem that we defer to future work; Section 5 discusses why it is deliberately out of scope for the present revision.

During replay, the system checks for a valid credential covering each event's timestamp; if found, the operation applies, otherwise it is skipped.

Trust operations (IssuerKey, IssuerKeyRevoke, StatusListChunk) build the TrustView incrementally during replay. When a VC must be checked, replicas parse the JSON Web Token (JWT), look up the issuer key in TrustView, verify the Ed25519 signature, check status-list revocation, and extract claims, all from in-band log state, with no online PKI lookups.

Unlike the W3C Bitstring Status List specification, which assumes HTTP-accessible status endpoints, EQUACK propagates status-list updates as in-band TRUST events in the causal DAG. This sidesteps the need for always-online StatusList URLs, status freshness is bounded by event propagation latency rather than HTTP polling intervals. The trade-off is that status information is only as fresh as the latest TRUST event a replica has received. EQUACK's design is compatible with W3C Bitstring Status List at the data-format level; deployments with partial connectivity can supplement in-band propagation with periodic HTTP fetches when available.

Implementation: Epoch construction is implemented in `core::epoch`; VC parsing and Ed25519 signature verification in `core::vc`. TrustView assembly, materializing issuer keys and status-list evidence from replayed TRUST events, is implemented in `core::trust` and `core::trustview`. Status-list bitstrings are verified using SHA-256 checksums.

3.3 Policy Evaluation and Deny-Wins Replay

EQUACK uses a policy layer inspired by AWS's Cedar language (open-source ABAC). Policies may permit or forbid actions with a default deny stance; deny-overrides semantics ensure any matching forbid overrides all permits. The framework unifies RBAC and ABAC, roles are principal at-

tributes, and Cedar's entity-attribute model supports hierarchical roles and contextual attributes. Policies are evaluated deterministically at each event's timestamp; identical inputs guarantee identical outcomes across replicas.

A REVOKE event deterministically terminates any authorization epoch opened by a prior or concurrent GRANT, so earlier permits cannot "win" due to message reordering. Revocation is not irreversible: a GRANT *after* a REVOKE in the replay order can open a fresh epoch under a different (re-issued) credential. For concurrent GRANT/REVOKE pairs, the deterministic linearization chooses a single order, and deny-wins applies *per epoch*, not per subject.

Concretely (illustrated in Figure 4), consider a concurrent GRANT g and REVOKE r for the same subject and scope, linearized as $\mathrm{pos}(r) < \mathrm{pos}(g)$. **Case A** (re-grant under stale credential): g references the same credential c_1 whose status-list entry r closes; the VC check fails at $\mathrm{pos}(g)$ and no epoch opens. **Case B** (re-grant under fresh credential): g references a new credential c_2 not covered by r's status update; the VC check succeeds and a fresh epoch opens at $\mathrm{pos}(g)$. Deny-wins is therefore enforced through two complementary paths – scope-based epoch closure at the REVOKE's position and credential-level status-list checks at VC verification time – without forbidding legitimate re-grants.

Our compliance fixture contains an in-band `IssuerKey`, credential-and-grant pairs for two writers, 50 concurrent pre-revoke writes from each writer, a merge point, a `REVOKE` for one writer, and 10 post-revoke writes from each writer. Replay first builds the in-band TrustView, verifies both credentials, and opens epochs for both writers. The later `REVOKE` closes only writer B's epoch in the deterministic replay order. As a result, all replicas apply both writers' pre-revoke writes, skip writer B's post-revoke writes, and continue applying writer A's writes.

Cedar's formal verification [27] establishes decidability, totality, and soundness of authorization evaluation under a single consistent context where all entity data and policies are available and coherent at decision time. In EQUACK's eventually consistent setting, this assumption holds locally (each replica evaluates deterministically given its local state) but not globally during partitions; two replicas may reach different permit/deny outcomes for the same request if one has received a forbid policy or revocation that the other has not. The deny-wins replay mechanism restores global consistency after convergence, once all replicas hold the same event set, replay re-evaluates every operation under the converged policy set, and the final materialized state $s^\star$ satisfies Cedar's deny-overrides semantics. Cedar's verified properties thus hold for $s^\star$ but not for the transient local decisions made during partitions. Formal verification of the replay mechanism itself, specifically that deterministic linearization plus deny-wins gating preserves the intended authorization invariants is future work and would require extending Cedar's Lean 4 proofs or independent mechanized checking.

The current model uses deny-overrides as the sole combining algorithm, any matching `forbid` policy overrides all `permit` policies. This is a deliberate safe default for multi-authority settings where no single authority should be able to override another authority's denial. Priority-based or temporal-precedence combining (e.g., last-writer-wins for policy updates) would require extending the epoch model to track policy precedence across authorities; we leave this for future work.

Entity attributes used in Cedar policy evaluation (e.g., role assignments, group memberships) may be stale during partitions, a replica that has not yet received an attribute-update event evaluates policies against outdated entity state. Because attribute changes are events in the causal DAG, they are replayed deterministically on convergence, and the final state $s^\star$ reflects the correct attribute values. Transient local decisions based on stale attributes are corrected during replay.

Concrete policy example: The Cedar-style policies the prototype evaluates against the experimental fleet workload follow the shape shown in Listing 1. The role binding (`maintenance_engineer`) and the resource scope (`compressor_config`) are carried by the issued credential as the `role` claim and the `scope` claim respectively; the `forbid` clause illustrates the deny-overrides combining algorithm that protects the gate from a stale or compromised attribute store.

```
permit(
  principal in Role::"maintenance_engineer",
  action == Action::"write",
  resource in Tag::"compressor_config"
) when {
  context.shift == "day" &&
  resource.safety_level < 3
};

forbid(
  principal,
  action == Action::"write",
  resource
) when {
  principal.certification_expired == true
};
```

Listing 1. Concrete Cedar-style policy used in the fleet experiments. The implemented evaluator supports the highlighted constructs (permit/forbid, role/scope matching, attribute equality, deny-overrides); the rest is illustrative for the discussion in Section 5.

Implemented Cedar subset: The prototype's evaluator is purpose-built (`crates/core/src/policy.rs`, ∼1,000 LOC including the epoch index, the deny-wins gate, and reason classification). It is not the upstream Cedar SDK. We therefore enumerate exactly which Cedar concepts the prototype implements and which it does not, so reviewers can match Listing 1 against the actual decision procedure:

- **Implemented:** `permit/forbid` effects with deny-overrides semantics; principal binding by Ed25519 public-key entity (`author_pk`); action discrimination across the three internal actions `SetField/SetAdd/SetRem` parsed from the data-op key prefix; static resource-tag inference (`tags_for`); scope intersection between credential scope and resource tags (`is_disjoint` check on `TagSet`); time-bound enforcement against the op's HLC via the credential's `nbf/exp` claims; topo-position bounds via the epoch's `[start_pos, end_pos)` window; deny classification with reason codes (`RevokedCred`, `ExpiredCred`, `OutOfScope`, `GenericDeny`) returned

to the audit hook; and a fixed role → permission table for the `editor` and `issuer_admin` roles.

- **Not implemented:** Cedar's full type system and schema validation; `unless` clauses; partial evaluation; the `when` clause grammar (boolean expressions over `context` and `resource` attributes such as Listing 1's `context.shift == "day"` and `resource.safety_level < 3` are interpreted at the policy-author level and lowered into static tag membership before they reach the evaluator); arbitrary entity hierarchies beyond the role/tag pairs registered in `role_perms/tags_for`; policy templates; and Cedar's Lean 4-verified soundness/decidability proofs (we re-derive the relevant deny-wins property informally in Section 3.7, Theorem 3).

The prototype scope is therefore close to ABAC-with-static-tags rather than full Cedar. Migrating to the upstream `cedar-policy` Rust SDK is a planned drop-in replacement (Section 5, "Cedar integration roadmap"); the current evaluator predates the Rust SDK's stabilisation and is retained because it is small, deterministic, and easy to wire into the deny-wins gate. The two design knobs that would change in a Cedar-SDK port are (1) replacing `role_perms/tags_for` with a Cedar policy set parsed at boot, and (2) replacing the bespoke `is_permitted_at_pos_with_reason` call with a Cedar `Authorizer` call whose `Decision` we map onto the existing `DenyDetail` enum.

Implementation: The deny-wins gate is implemented in `core::policy` (`is_permitted_at_pos_with_reason`). Full replay (`core::replay::replay_full`) and incremental replay from checkpoints (`core::replay::apply_incremental`) both apply the gate as a pure fold over the linearized sequence. Policy evaluation uses a purpose-built evaluator covering the Cedar subset enumerated above. Figure 4 shows this replay slice and the epochs and gate outcomes it induces.

3.4 Tamper-Evident Audit Trail

During replay, each replica emits a signed *audit record* per DATA operation indicating applied/skipped status and reason. Records are linked in an append-only hash chain; any inconsistency is detectable relative to a pinned chain head digest (e.g., cross-signed checkpoint). Because replay is deterministic, independent replicas with the same event set compute identical audit traces and can validate each other after reconciliation (R4).

Implementation: Audit chain management is implemented in `store::audit`. The `replay_full_with_audit` entry point accepts an `AuditHook` trait object, emitting signed records during replay. Checkpoints (state snapshot plus topological watermark) are managed by `store::checkpoint` backed by RocksDB.

3.5 Confidentiality and Key Rotation

EQUACK supports an optional per-tag payload encryption layer. Each DATA operation carries ciphertext $c := \mathsf{Enc}_{K_\tau^{(v)}}(m)$ with key version v (XChaCha20-Poly1305). Ciphertext is replicated for convergence; access control is enforced at *query time* via the deny-wins gate, with unauthorized subjects receiving a redacted view. On revocation, a KeyRotate event advances to $K_\tau^{(v+1)}$, distributed in-band via KeyGrant records encrypted to authorized subjects' public keys. This provides *leaver security*; revoked subjects lacking $K_\tau^{(v+1)}$ cannot decrypt post-rotation ciphertexts. This component is experimental; we treat confidentiality results as indicative.

Implementation: Encryption and decryption use XChaCha20-Poly1305 in `core::crypto`. Per-tag key versioning and KeyGrant/KeyRotate handling use X25519 key agreement without a full ceremony protocol. Tag keys are persisted by `store::put_tag_key` / `store::get_tag_key`.

3.6 Formal Model and Guarantees

Let Subj be subjects (principals), Res resources, Act actions, and Attr attribute valuations. Physical time is measured in milliseconds as $(T_{\mathrm{ms}} = \mathbb{N}, \leq)$. Let NodeId be replica identifiers, and let hybrid logical clock timestamps range over $T_{\mathrm{hlc}} := T_{\mathrm{ms}} \times \mathbb{N} \times \mathsf{NodeId}$ ordered lexicographically. Let $\mathrm{phys} : T_{\mathrm{hlc}} \to T_{\mathrm{ms}}$ project the physical millisecond component. We use T_{ms} for credential validity intervals and status freshness, and T_{hlc} (via lexicographic order) only for deterministic linearization. Let H be a collision-resistant hash and $\mathsf{Sig}_k(\cdot)$ a secure signature scheme.

A verifiable credential (VC) is a tuple

$$c = \langle s,\, \alpha,\, V,\, \mathrm{nb},\, \mathrm{na},\, \sigma_I \rangle$$

where $s \in \mathsf{Subj}$, $\alpha \in \mathsf{Attr}$ (roles/scopes), V issuer identifier, and $\mathrm{nb}, \mathrm{na} \in T_{\mathrm{ms}}$ are not-before/after bounds. σ_I is a signature under issuer key K_V. A status list is a signed, time-versioned snapshot published by issuer V that maps credential identifiers to $\{\mathsf{good}, \mathsf{revoked}\}$. We model status evidence as a sequence of TRUST events that carry status-list updates (or chunks). Let SL_V^i denote the status map materialized by replaying all TRUST events with $\mathrm{pos}(e) \leq i$ and taking the latest published status for each credential id. When validating a credential for an event e, we query $\mathsf{SL}_{\mathrm{iss}(c)}^{\mathrm{pos}(e)}(\mathrm{id}(c))$. A *TrustView* event carries (i) issuer keys $\{K_V\}$ and (ii) status-list updates, recorded in-band so replicas can validate VCs offline using the latest status snapshot up to position $\mathrm{pos}(e)$ in the replay order.

Events are partitioned as

$$\mathsf{Ev} = \mathsf{DATA} \uplus \mathsf{GRANT} \uplus \mathsf{REVOKE} \uplus \mathsf{TRUST} \uplus \mathsf{KEY}.$$

The subset KEY contains key-management events (e.g., KEYROTATE, KEYGRANT) that update per-tag key state used for query-time decryption. Each event $e \in \mathsf{Ev}$ has an *unsigned* header $h_0(e)$ containing domain fields, causal parents $\mathsf{par}(e) \subseteq \mathsf{Ev}$, a hybrid logical clock $\mathrm{hlc}(e) \in T_{\mathrm{hlc}}$ assigned by the producing replica, the producer public key pk_r, and the payload (or a payload hash). We define the event identifier as

$$\mathsf{op_id}(e) := \mathsf{H}(\mathrm{domain} \,\|\, \mathrm{CBOR}(h_0(e))).$$

The producing replica signs this identifier:

$$\sigma_e := \mathrm{Sig}_{\mathrm{sk}_r}(\mathsf{op_id}(e)).$$

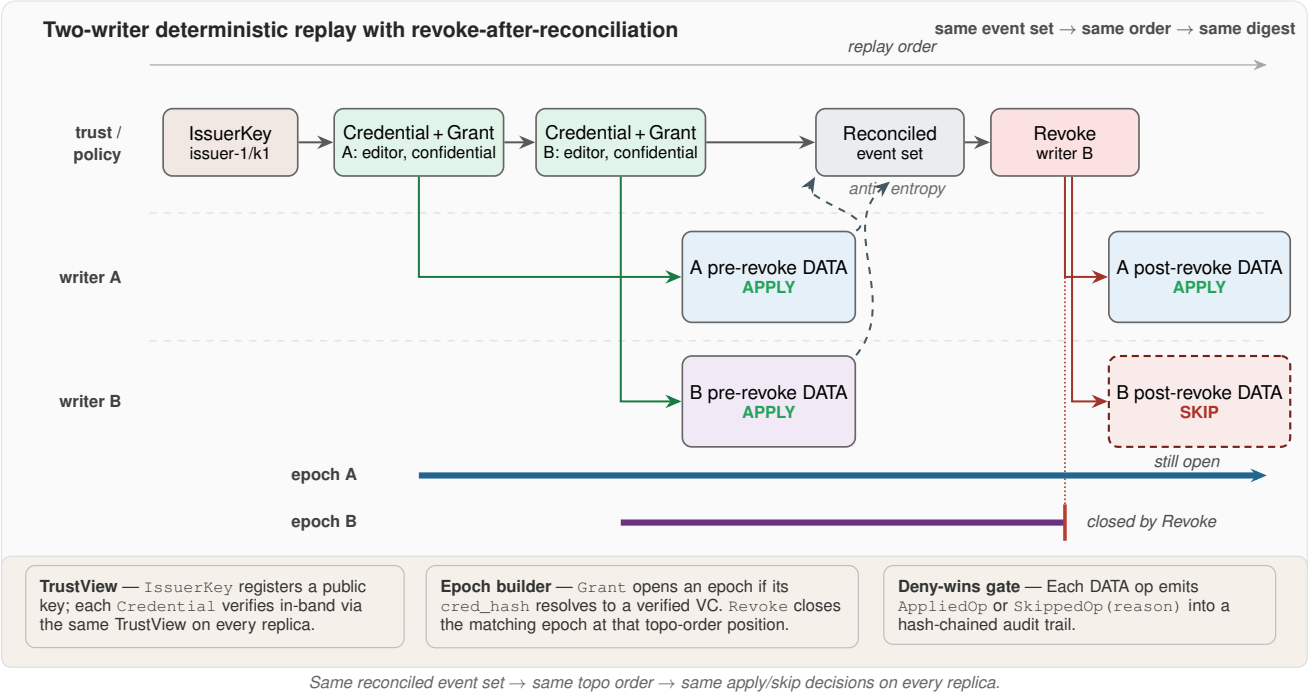

Fig. 4. Two-writer replay example under deterministic deny-wins semantics. An in-band `IssuerKey` seeds TrustView; verified `Credential`+`Grant` pairs open epochs for writers A and B; reconciliation yields one shared event set; and `Revoke(B)` closes only B's epoch in the deterministic replay order. Every replica therefore applies both writers' pre-revoke data, skips B's post-revoke data, continues applying A's post-revoke data, and emits the same audit outcomes.

Signing $\mathsf{op_id}(e)$ is sufficient because $\mathsf{op_id}(e)$ commits to the canonical bytes of $h_0(e)$ under collision resistance of H. The full header is $h(e) := (h_0(e), \sigma_e)$. This avoids any circularity; $\mathsf{op_id}(e)$ is derived from the unsigned header, and σ_e covers $\mathsf{op_id}(e)$. The *signed, hash-linked op log* is an append-only bag of events with edges $e' \to e$ for $e' \in \mathsf{par}(e)$. The induced graph $G = (\mathsf{Ev}, E)$ must be a DAG; this causal DAG defines the partial order $\prec$ as the transitive closure of E.

Let the deterministic key be

$$k(e) := \big(\mathrm{hlc}(e), \mathsf{op_id}(e)\big),$$

ordered lexicographically. Define the replay sequence $\langle e \rangle_{\triangleleft}$ as the output of Algorithm 1 applied to the causal DAG G. We then define $e_i \triangleleft e_j$ iff e_i appears before e_j in $\langle e \rangle_{\triangleleft}$. Let $\langle e \rangle_{\triangleleft} = [e_0, \ldots, e_{n-1}]$ be the output of Algorithm 1. Define the deterministic replay position $\mathrm{pos}(e_i) := i$.

If two events e_A and e_B are concurrent and both are ready (zero indegree) at some step, Algorithm 1 selects the smaller key, if $\mathrm{hlc}(e_A) < \mathrm{hlc}(e_B)$ then e_A is chosen first; if equal, the smaller op_id is chosen.

Epochs are computed deterministically by replaying non-DATA events in $\langle e \rangle_{\triangleleft}$. A GRANT event g at position $p_g := \mathrm{pos}(g)$ that references a valid credential opens an epoch record $\mathsf{ep} = \langle s, a, \mathrm{scope}, p_{\mathrm{start}}{=}p_g, p_{\mathrm{end}}{=}\infty, \mathrm{nb}, \mathrm{na} \rangle$. A REVOKE event r at position $p_r := \mathrm{pos}(r)$ closes matching open epochs by setting $p_{\mathrm{end}} := \min(p_{\mathrm{end}}, p_r)$. We write $\mathsf{ep} \in \mathsf{Epochs}(s, a)$ for epochs associated with subject s and action a.

Algorithm 1 Deterministic linearization of a causal DAG

Require: DAG $G = (V, E)$, key $k(e) = (\mathrm{hlc}(e), \mathsf{op_id}(e))$
Ensure: Sequence L that is a deterministic linear extension of $\prec$

```
indeg[v] ← |{(u, v) ∈ E}| for all v ∈ V
Q ← min-priority queue ordered by k(·)
for all v ∈ V with indeg[v] = 0 do
    Q.push(v)
end for
L ← [ ]
while Q not empty do
    v ← Q.pop_min()
    append v to L
    for all (v, w) ∈ E do
        indeg[w] ← indeg[w] − 1
        if indeg[w] = 0 then
            Q.push(w)
        end if
    end for
end while
return L
```

For subject s, scope $a \in \mathsf{Act}$, and event e, define the authorization predicate

$$\mathsf{Auth}(s, a, e) := \exists c \text{ s.t. } \mathrm{phys}(\mathrm{hlc}(e)) \in [\mathrm{nb}, \mathrm{na}) \;\wedge\; c \text{ grants } (s, a) \;\wedge\; \mathsf{SL}^{\mathrm{pos}(e)}_{\mathrm{iss}(c)}(\mathrm{id}(c)) = \mathsf{good}.$$

Status-list Time-to-Live (TTL) is a freshness hint for operational policy; our determinism guarantees assume replicas replay the same TrustView history, while deployments may

additionally deny decisions when status evidence is older than a configured threshold.

Clock skew and validity boundaries: The authorization predicate $\mathsf{Auth}(s, a, e)$ evaluates credential validity using $\mathrm{phys}(\mathrm{hlc}(e))$, the physical millisecond component of the event's HLC, which is recorded by the producing replica at event-creation time. Under the bounded clock skew assumption (Section 2.2, item 7), this recorded timestamp is within ϵ of real time. Consequently, an event close to a credential's nb or na boundary may be classified differently than intended: a replica whose clock is skewed by $\delta \leq \epsilon$ from real time records a physical component off by δ, so events within ϵ of a boundary may be accepted on one replica and rejected on another during the partition. Setting credential validity margins larger than ϵ eliminates this ambiguity for events that are well inside the validity interval. Crucially, deterministic convergence (Theorem 1) is *unaffected* by skew: after reconciliation, all replicas evaluate the same $\mathrm{hlc}(e)$ against the same credential bounds and produce the same verdict. Determinism depends on the HLC values recorded in the log, not on their accuracy relative to real time; only the *intended* semantic correspondence between $\mathrm{phys}(\mathrm{hlc}(e))$ and real wall-clock time degrades with skew.

Let $\mathcal{P}$ denote the Cedar policy, modeled as a total function

$$\mathcal{P} : \mathsf{Subj} \times \mathsf{Act} \times \mathsf{Res} \times \mathsf{Attr} \to \{\mathsf{allow}, \mathsf{deny}\},$$

with *deny-overrides* semantics. For a DATA event e with tuple (s, a, r, α_e), define the *gate function*—the predicate that determines whether a data event is authorized at its position in the replay order:

$$\begin{aligned}\mathsf{Gate}(e) := \Big(&\exists\, \mathsf{ep} \in \mathsf{Epochs}(s, a) \text{ s.t.} \\ &\quad \mathrm{pos}(e) \in [p_{\mathrm{start}}, p_{\mathrm{end}}) \wedge \\ &\quad \mathrm{phys}(\mathrm{hlc}(e)) \in [\mathrm{nb}, \mathrm{na})\Big) \\ &\wedge\ \mathcal{P}(s, a, r, \alpha_e) = \mathsf{allow}.\end{aligned}$$

If *any* deny applies, $\mathcal{P}$ returns deny and the DATA op is skipped (deny-wins).

Let $(\mathcal{S}, \bullet)$ be the state space with a deterministic update function $U : \mathcal{S} \times \mathsf{DATA} \to \mathcal{S}$. We assume the DATA payloads target CRDTs such as MVReg and $\mathrm{OR-Set}$ whose effect under the same total order is deterministic. Given initial state s_0, define *deterministic replay* over the sequence $\langle e \rangle_{\lhd}$:

$$\begin{aligned}\mathsf{Replay}(s_0, \langle e \rangle_{\lhd}) := \mathrm{fold}\Big(&s_0,\ \lambda(s, e). \\ &\begin{cases} U(s, e) & \text{if } e \in \mathsf{DATA} \ \wedge\ \mathsf{Gate}(e) \\ s & \text{otherwise} \end{cases}\Big).\end{aligned}$$

The *materialized state* (the computed state after replaying all events) is $s^\star := \mathsf{Replay}(s_0, \langle e \rangle_{\lhd})$. Checkpoints store $(\mathsf{H}(s^\star), \mathrm{meta})$ periodically together with the append-only event store.

Define the audit stream as a sequence of records

$$\begin{aligned}\mathsf{Aud}(i) &= \langle e_i,\ \mathsf{Gate}(e_i),\ \mathsf{H}(s_i) \rangle, \\ \mathrm{ch}_0 := \bot, &\quad \mathrm{ch}_{i+1} := \mathsf{H}\big(\mathrm{ch}_i \,\|\, \mathsf{Aud}(i)\big),\end{aligned}$$

signed by the auditing replica (producer) to yield a tamper-evident chain (ch_i) of applied/skipped decisions.

Let Tag denote data tags; each tag τ has a versioned symmetric key $K_\tau^{(v)}$. A DATA payload m is stored as (c, v) where $c := \mathsf{Enc}_{K_\tau^{(v)}}(m)$. Ciphertexts are replayed and replicated for convergence; authorization controls *decryption* at query time. On a KeyRotate event for τ at time t_r, the current key version advances to $v{+}1$ and subsequent writes use $v{+}1$; KeyGrant records distribute $K_\tau^{(v+1)}$ only to subjects authorized to read τ after t_r.

Events are disseminated via secure anti-entropy gossip; missing parents are fetched until G is complete. This affects *when* a replica can replay but not *what* $s^\star$ is, once the same set of events is present.

Pre-convergence safety window: Let r be a revocation event created at physical time t_r on some replica. Define the *safety window* $W(r)$ as the interval $[t_r, t_{\mathrm{heal}}]$ where t_{heal} is the earliest time at which all correct replicas have received r via anti-entropy. During $W(r)$, replicas that have not yet received r may accept operations from the revoked principal under stale authorization evidence. Under the bounded partition assumption (Section 2.2, item 5), $W(r) \leq \Delta_{\mathrm{partition}} + \Delta_{\mathrm{propagation}}$ is finite.

EQUACK does not prevent tentative authorization during $W(r)$; it guarantees that the materialized state $s^\star$ computed after reconciliation reflects r. Specifically, any DATA event e from the revoked subject with $\mathrm{pos}(e) \geq \mathrm{pos}(r)$ in the deterministic order $\lhd$ is skipped by the deny-wins gate, regardless of which replica originally accepted e. This provides *retroactive revocation safety*, the final digital state is consistent with the revocation, even though individual replicas may have temporarily acted on stale evidence. In contrast, Zanzibar's Zookie mechanism [26] provides real-time revocation at the cost of availability during partition (consistent with Consistency–Availability–Partition tolerance (CAP) theorem constraints). EQUACK trades real-time revocation for availability, repairing digital-state safety on merge (physical-world consequences are the application's responsibility; see Section 5).

3.7 Main Theorems

R1–R6 in Section 2.3 are *system-level requirements* (deployability and security objectives) for offline-first, multi-stakeholder settings. The theorems below are *formal guarantees* about EQUACK's deterministic replay semantics and audit/confidentiality mechanisms under the model assumptions. The gap analysis (Table 1) maps R1–R6 to paradigm coverage; the theorems below formalize the guarantees, and the evaluation (Section 4) provides empirical evidence.

The guarantees below are model-level properties of the deterministic replay semantics. They hold under the assumptions stated in Section 2.2, assuming correct implementation of the replay and gate logic. The prototype (Section 4) provides empirical evidence, but the replay-gate composition has not been mechanically verified; the guarantees should therefore be understood as formal claims supported by proof sketches and prototype testing, not as end-to-end verified system properties. Mechanized verification of the replay-gate composition is a priority for future work.

Theorem 1 (Determinism). *For any DAG G and initial state s_0, the order $\lhd$ is unique and* $\mathsf{Replay}(s_0, \langle e \rangle_{\lhd})$ *yields a unique $s^\star$.*

Proof sketch. Two-step induction. (i) On DAG structure: Algorithm 1 is Kahn's topological sort with a deterministic min-heap keyed by $(\text{HLC}, \text{key id})$ over the ready set. By induction on $|G|$, at each step the heap admits a unique minimum (keys are total), so removing it leaves a smaller DAG with a unique extension; concatenation yields a unique linear extension $\langle e \rangle_{\triangleleft}$ for G. (ii) On fold position: Replay is a left fold $s_{i+1} = U(s_i, e_i)$ where U is a pure, deterministic function (CRDT merge plus gate-guarded effect). By induction on i, s_i depends only on s_0 and $e_0, \ldots, e_{i-1}$, so the final $s^\star = s_n$ is unique. □

Theorem 2 (Convergence). *Fix bootstrap anchors B consisting of a pinned, cross-signed genesis checkpoint digest (and the stakeholder public keys used to validate it). Let two replicas start from the same anchors B and ingest the same multiset of well-formed events (including* TRUST *and* KEY *events). Then their materialized states are byte-identical: $s_1^\star = s_2^\star$.*

Proof sketch. Follows from Theorem 1 plus the observation that identical bootstrap anchors and identical event multisets produce identical DAGs: each replica's parent set is recovered from the events themselves, and set union is commutative and idempotent, so $G_1 = G_2$. By Theorem 1, both replicas obtain the same linearization $\langle e \rangle_{\triangleleft}$, the same $\text{pos}(\cdot)$ assignments, the same epoch construction, and the same deny-wins gate decisions; applying the same ordered sequence of authorized DATA updates under deterministic U yields byte-identical $s^\star$. □

Theorem 3 (Policy-Safety, deny-wins). *Every effect in $s^\star$ originates from a* DATA *event e such that* $\mathsf{Gate}(e)$ *held at position* $\text{pos}(e)$*. If any deny applies (policy or missing epoch), e has no effect.*

Proof sketch. Induction on position i. At each step, $\mathsf{Gate}(e_i)$ is evaluated against the epoch table built from events $e_0, \ldots, e_{i-1}$ (Theorem 1 fixes this prefix uniquely). If any REVOKE at position $j < i$ closed the relevant epoch, then $p_{\text{end}} \leq j < i$, so $\text{pos}(e_i) \notin [p_{\text{start}}, p_{\text{end}})$ and the gate returns deny; the deny-wins combinator (Section 3.6) ensures this overrides any concurrent allow. By definition of Replay, e_i then produces no state transition. Hence every effect that survives in $s^\star$ originates from a DATA event whose gate held at $\text{pos}(e)$. □

Theorem 4 (Audit tamper-evidence). *Let ch_N be a pinned audit-chain head digest for the first N records (obtained out-of-band or via cross-signing). If the audit chain verifies and the computed head equals ch_N, then any modification to the first N audit records (deletion, insertion, reordering, or bit-flip) is detectable with overwhelming probability.*

The head digest is a collision-resistant hash of the prefix. Any change to the prefix changes the computed head unless the adversary finds a collision/second preimage or forges a signature.

Theorem 5 (Leaver security). *Fix a tag τ and a rotation event at time t_r that advances to key version $v{+}1$. Assume indistinguishability under chosen-plaintext attack (IND-CPA) encryption and that $K_\tau^{(v+1)}$ is delivered only to subjects authorized for read_τ at times $\geq t_r$. Then any subject not authorized for read_τ at times $\geq t_r$ cannot decrypt ciphertexts produced under version $v{+}1$, except with negligible probability.*

Proof sketch. By assumption, unauthorized subjects do not obtain $K_\tau^{(v+1)}$. Under IND-CPA security, ciphertexts under an unknown key reveal no information about plaintext beyond negligible probability. □

Scope and gap. This claim is conditional on the key-distribution assumption: that $K_\tau^{(v+1)}$ reaches exactly the post-rotation authorized set. Closing this gap requires a group-key-agreement ceremony (e.g., a Messaging Layer Security style protocol) that the paradigm does not itself supply. The EQUACK prototype models distribution with X25519 point-to-point envelopes per authorized subject (Section 4); this is sufficient to measure rotation cost (E14) but does not constitute a deployable ceremony. A production deployment would substitute a group-key-agreement protocol; integrating one and re-stating Theorem 5 with its concrete adversary model is future work (Section 4.4).

Table 3 summarizes the mapping from conceptual elements to *equack-core* modules. The prototype is organized into four crates: `core` (DAG, replay, policy, VCs, CRDTs), `store` (RocksDB persistence, checkpoints, audit), `net` (libp2p gossip and anti-entropy), and `cli` (workload generation and experiment execution).

TABLE 3
Mapping from conceptual model to implementation modules.

Conceptual Element	Implementation Module
Causal DAG $(G, \prec)$	`core::dag`
Deterministic order $(\triangleleft)$	`core::replay::topo_sort`
Authorization epochs	`core::epoch`
Deny-wins gate (Gate)	`core::policy::gate`
TrustView assembly	`core::trust`
VC verification	`core::vc`
CRDT state $(\mathcal{S}, U)$	`core::crdt`
Audit chain	`store::audit`
Checkpoints	`store::checkpoint`
Per-tag encryption	`core::crypto`
Gossip/sync	`net::gossip`

TABLE 4
Experiment summary. T1–T5 refer to the theorems in Section 3.7; "Perf" denotes performance characterization without a corresponding theorem.

ID	Experiment	Theorems
E1	Convergence (100 orderings)	T1, T2
E2	Replay determinism regression	T1
E3	Deny-wins revocation	T3
E4	Multi-authority conflict	T1–T3
E5	Audit integrity (3 tamper modes)	T4
E6	Replay scaling + checkpoints	Perf
E7	Throughput by workload	Perf
E8	Storage growth	Perf
E9	Peak memory (RSS)	Perf
	E10 (checkpoint speedup) merged into E6.	
E11	Partition-heal revocation	T2, T3
E12	Multi-node sync	T2, T3
E13	Encryption overhead	T5
E14	Key rotation cost	T5
E15	Edge-device replay (aarch64)	Perf
E16	Networked multi-node sync	T2, T3
E17	IoT fleet workload	Perf

4 EVALUATION

We evaluate EQUACK through deterministic experiments that provide empirical evidence for the main theorems (Theorems 1–5, Section 3.7) and quantify replay cost. Table 4 maps each experiment to the theorem(s) it targets. All experiments use fixed seeds and a deterministic environment (`TZ=UTC`, `SOURCE_DATE_EPOCH=1`). Offline operation without continuous global consensus is a precondition of the model, not an experimental variable.

4.1 Methodology

Experiments ran on a single Linux x86_64 workstation using Rust 1.85 in release mode. Multi-node convergence (E12) uses in-process deterministic simulation; partition behavior is validated via offline log merges. State digests are computed as $\mathrm{blake3}(\mathrm{DOMAIN} \,\|\, \mathrm{canon}(\textit{state}))$, where $\mathrm{canon}(\cdot)$ is a deterministic byte serialization with stable key ordering.

We use three synthetic workloads (Table 5) and two policy-focused CBOR logs. The *hb-chain* workload is a single writer appending sequentially to one key, giving a baseline with no concurrency or policy overhead. The *concurrent* workload has eight writers modifying the same key, stressing the MVReg CRDT's conflict resolution. The *offline-revocation* workload mixes writes with mid-workload revocations, forcing epoch recalculation during replay. Two additional logs exercise deny-wins behavior; `e3-policy.cbor` (grant, 500 writes, revoke, 500 writes) and `e4-policy.cbor` (multi-issuer grants, revoke, post-revoke writes). For the partition scenario, logs `partition-A.cbor` and `partition-B.cbor` were generated and merged to model revocation propagation on reconnection.

TABLE 5
Synthetic Workload Characteristics

Workload	Writers	Policy Ops	Key Dist.	Purpose
hb-chain	1	0%	Single	Baseline
concurrent	8	0%	Single	CRDT stress
offline-revocation	8	Mixed	Single	Policy overhead
iot-fleet	20	Mixed	Multi	Domain scenario (E17)

Sections 4.2 and 4.3 present correctness experiments (E1–E5, E11–E12) that validate the formal claims, followed by performance experiments (E6–E9, E13–E14) that characterize replay cost, storage, and encryption overhead.

4.2 Correctness

Convergence (E1; T1, T2): A 10,000-op concurrent-workload log was permuted into 100 valid orderings (respecting causal dependencies) and each was replayed independently. All 100 produced the same digest (`cbab89ee...b7f493b2`), with mean replay time 615 ms ($\sigma = 18$ ms). Delivery order does not affect the final authorized state; a prerequisite for safe offline operation across independently administered replicas.

Deny-wins revocation (E3; T3): User U was granted write access, performed 1,000 writes, and had the credential revoked after write 500. The 1,004-op log (1,000 Data + IssuerKey, Credential, Grant, Revoke) was replayed; the first 500 writes applied, the remaining 500 skipped. Digest: `8da8b49e...46633fbf`. An engineer whose credential is revoked after a safety incident has all post-revocation parameter changes excluded from the converged state, regardless of which controllers received the revocation first.

Revocation under partition (E11; T2, T3): In partition A a user is revoked; partition B, which has not yet received the revocation, contains further writes from the same user under stale evidence. After merging both logs and replaying, the 50 pre-revocation data writes (from both partitions) were applied and all 10 post-revocation writes from partition B were skipped. A revocation issued on one side of a partition retroactively invalidates stale writes accepted on the other side.

Multi-authority conflict (E4; T1, T3): Three issuers (I_1, I_2, I_3) with overlapping authority grant user U the "editor" role at t_1, t_2; I_3 revokes at $t_3 > t_2$. The resulting 128-op log was injected into 10 replicas in random orders. All converged to the same digest (`f2fbb024...1103c2b2`) writes during $[t_1, t_3)$ accepted, writes after t_3 skipped.

Audit integrity (E5; T4): An audited log was generated and the chain head digest ch_N was recorded. Three tamper modes were applied; (1) a byte flip in one record's payload, (2) deletion of one record from the pinned prefix, and (3) a swap of two adjacent records. Chain verification against ch_N succeeded on the untampered log and failed on all three mutations, detecting each at the exact tampered record. Any party can independently verify that no authorization decisions were altered after the fact.

Replay determinism regression (E2; T1): A 127-op CBOR fixture exercises all event categories (Data, Grant, Revoke, Trust) and includes true concurrency. A Continuous Integration (CI) test replays it on one Linux x86_64 target and asserts the final digest equals a hardcoded constant (`4ea9684a...bdc2b9`, blake3). Any replay-engine change that alters the digest breaks the build; this serves as a regression guard on the deterministic linearization and gate logic rather than as a cross-platform validation. We additionally verified that the Raspberry Pi 4 (aarch64) run of the IoT-fleet workload (E17) produced a digest byte-identical to the x86_64 reference for the same seed, confirming cross-platform determinism across ISAs.

Multi-node sync (E12; T2, T3): In an in-process simulation, three nodes each generated 1,000 ops independently, then synced; all converged. A second scenario partitioned node C while A issued a Revoke targeting C. After healing and full sync, all three nodes held the same digest, with C's post-revoke ops gated out (Table 6). This demonstrates end-to-end deny-wins enforcement across independently operating nodes.

4.3 Performance

Replay scaling (E6): Hb-chain logs at six sizes (10K–500K ops) were each replayed in 5 trials, measuring both full replay and incremental replay from a 90% checkpoint (Table 7).

TABLE 6
Multi-node sync convergence (E12)

Scenario	Ops	Sync (ms)	Bytes	Msgs	Replay (ms)
No partition	3,000	13	2.5 MB	6,000	411
Partition + heal	3,000	7	1.3 MB	3,201	368

TABLE 7
Replay time vs. log size (mean ± std, 5 trials)

Ops	Full (ms)	Incr. (ms)	Speedup
10,000	949 ± 395	98 ± 37	9.7×
20,000	$1{,}294 \pm 79$	147 ± 21	8.8×
50,000	$3{,}328 \pm 233$	404 ± 54	8.2×
100,000	$6{,}071 \pm 92$	708 ± 86	8.6×
250,000	$13{,}689 \pm 1{,}814$	$1{,}644 \pm 232$	8.3×
500,000	$26{,}616 \pm 2{,}091$	$3{,}137 \pm 310$	8.5×

Full replay scales linearly (≈0.052 ms/op, $R^2 > 0.99$), giving ≈19K ops/s sustained throughput on the hb-chain workload. Incremental replay from a 90% checkpoint yields 8–11× speedup across all sizes; the factor is stable because checkpoint restore is $O(1)$ and the suffix cost is proportional to 10% of the log. Checkpoint creation overhead is under 10 ms.

Throughput by workload (E7): Throughput varies with workload complexity (Table 8). Linear chains achieve the highest rate; concurrent writes reduce throughput because of MVReg conflict resolution; policy-heavy workloads show modest epoch-construction overhead.

TABLE 8
Replay throughput by scenario

Scenario	Throughput (ops/s)
hb-chain (linear)	45,000
concurrent (8 writers)	7,000
offline-revocation	28,000

Figure 5 visualizes the scaling trends quantified by Tables 7 and 8; full replay is approximately linear, checkpointed replay is consistently faster by about one order of magnitude, and workload concurrency materially reduces throughput.

Phase profile (8-writer concurrent): To attribute the ≈6× slowdown of the concurrent scenario relative to `hb-chain`, we instrumented the replay pipeline phase by phase using `Instant::now()` (`bench_profile`) on a 10K-op concurrent log[2]. The dominant cost is the MVReg HB-aware put loop (≈4,147 ms, 96.9% of `replay_full`), followed by Ed25519 signature verification (426 ms, 10.0%); DAG insertion (6.4 ms), Algorithm 1 topological sort (3.0 ms), and the terminal state digest (13 μs) are collectively under 0.3% (Table 9). Doubling the workload from 5K to 10K ops increases the MVReg phase from 975 ms to 4,147 ms (≈4.25×), confirming that the bottleneck is the per-put `dag_is_ancestor` traversal, which currently re-walks the ancestry stack from each prior winner; an op-position cache or precomputed ancestor index would convert this loop to amortized linear and is a natural follow-on optimization.

2. The host runs with `kernel.perf_event_paranoid=4`, so `cargo flamegraph/perf record` are unavailable; the instrumentation bench reproduces the same phases as `apply_over_order` and times each in isolation. Numbers are wall time in microseconds, single trial, release build.

TABLE 9
Replay phase breakdown on the 8-writer concurrent workload (10,000 ops, single trial, release build, microseconds; percentages are of `replay_full`).

Phase	Wall (μs)	% of replay_full
DAG insert	6,389	0.15%
Topo sort (Alg. 1)	2,983	0.07%
Ed25519 verify	426,017	9.95%
MVReg HB-aware put	4,147,477	96.90%
State digest	13	0.00%
`replay_full`	4,280,045	100.00%

Storage and memory (E8, E9): CBOR log size grows linearly at ≈430 bytes/op; RocksDB write amplification brings the on-disk cost to ≈800 B/op (Table 10). Checkpoint state remains under 200 bytes for the single-key workload. Peak Resident Set Size (RSS) during replay grows at ≈1.7 KB/op (Table 11), dominated by the in-memory DAG and CRDT state.

TABLE 10
Storage growth vs. op count

Ops	CBOR Log	Ckpt State	Store Total	B/op
10,000	4.1 MB	122 B	7.7 MB	804
50,000	20.6 MB	118 B	37.8 MB	792
100,000	41.4 MB	127 B	75.7 MB	793
500,000	208.9 MB	130 B	555.8 MB	1,165

TABLE 11
Peak RSS during replay

Ops	Peak RSS (MB)	Replay (ms)
20,000	31	1,331
100,000	131	6,571
250,000	421	34,622

Confidentiality overhead (E13, E14; T5): Per-tag encryption adds ∼6.5% replay overhead (76.8 vs. 81.8 ms on 100 writes) with 40 bytes/payload ciphertext expansion. Key rotation costs ∼78 ms per KeyRotate event. These are small-scale measurements; larger logs are needed to separate per-op cost from startup overhead.

Asymptotic cost: Let $n = |\mathsf{Ev}|$ be the number of events and $m = |E|$ the number of causal DAG edges. Algorithm 1 (deterministic linearization) runs in $O(n \log n + m)$: each event is inserted into and extracted from the min-heap exactly once ($O(n \log n)$), and each edge is traversed once ($O(m)$). Replay is a single pass over the linearized sequence: $O(n)$ gate evaluations, each requiring epoch lookup ($O(\log k)$ for k concurrently open epochs in a subject's scope) and Cedar-inspired policy evaluation ($O(p)$ for p policy rules). TrustView growth is $O(t)$ where t is the number of TRUST events (issuer keys and status-list chunks); in practice $t \ll n$. Checkpoint restore is $O(1)$ for the state

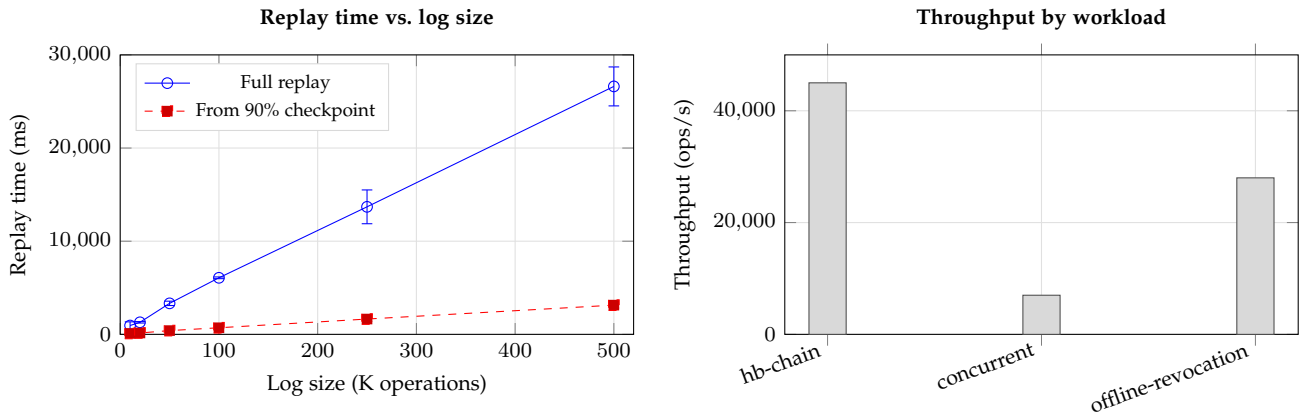

Fig. 5. Replay cost trends. Full replay scales approximately linearly with log size, while restoring from a 90% checkpoint reduces replay time by roughly one order of magnitude across the tested range. Throughput is highest on linear chains, falls under high concurrency because of MVReg conflict handling, and remains materially below the linear baseline under policy-heavy revocation workloads.

snapshot plus $O(n - n_{\text{ckpt}})$ for replaying the suffix, matching the near-constant-factor speedup observed in Table 7. Storage is $O(n)$ for the append-only log (approximately 430 bytes/op in CBOR, Table 10) plus $O(n)$ for RocksDB indices. Peak memory during replay is dominated by the in-memory DAG ($O(n + m)$) and CRDT state ($O(n)$ in the worst case; sub-linear when multi-value registers collapse conflicting writes).

Edge-device replay (E15; Perf): E15 recompiles the replay and throughput harnesses used in E6/E7 for an `aarch64-unknown-linux-gnu` target representative of an industrial edge gateway (Raspberry Pi 4-class, Cortex-A72, 4 GiB LPDDR4), cross-compiled via the toolchain declared in `.cargo/config.toml` and deployed through `deploy/run-edge-benchmark.sh`. The harness is deterministic (same fixed seeds as the x86_64 runs) so the digests produced on the edge device can be byte-compared against the workstation numbers to additionally confirm cross-platform replay determinism (strengthening E2 beyond a single ISA). Table 12 reports the edge-device timings for the same size matrix used in E6.

TABLE 12
Edge-device replay (E15) – Raspberry Pi 4 class, aarch64.

Ops	Replay (ms)	vs. x86_64	Digest match
10,000	3,600	3.8×	✓
50,000	18,026	5.4×	✓
100,000	36,156	6.0×	✓

Networked multi-node sync (E16; T2, T3): The multi-node sync experiment (E12) simulates the libp2p layer in-process by passing CBOR frames through a shared channel. E16 retains E12's workload but swaps the in-process channel for the real libp2p stack running in a three-container Docker Compose testbed (`deploy/docker-compose.yml`) driven by `deploy/run-network-experiment.sh`. Each run covers three conditions: (A) baseline (no delay, no loss), (B) a moderate WAN emulation applied on one node's uplink via `tc netem` (100 ms delay, 1% loss), and (C) a partition-and-heal cycle in which `inject-partition.sh` drops all traffic between two nodes for 30 s, during which node A issues a REVOKE targeting a subject still writing on the isolated node B. Convergence is measured as the wall-clock time from the moment all partitions heal to the moment every node reports the same state digest (`measure-convergence.sh`). E16 does not alter the model: the libp2p stack is an off-the-shelf transport and the replay/gate pipeline is byte-identical to E12. The experiment exists to confirm that the deterministic convergence argument (Theorem 1) and the deny-wins lift (Theorem 3) hold when frames traverse a real kernel network stack subject to reordering, loss, and queueing.

TABLE 13
Networked multi-node sync (E16).

Condition	Convergence (s)	Digest match
(A) baseline	4	✓
(B) 100 ms + 1% loss	3	✓
(C) partition + heal (30 s)	5	✓

E16 runs three Docker containers on a single host so that the `tc netem` qdisc can be applied deterministically and the experiment is reproducible from a single `docker compose` invocation. The reviewer (W1, 10 April 2026) correctly notes that this is not a multi-host deployment: every container shares the host kernel's loopback queues, so the test exercises libp2p's QUIC framing, the gossip and anti-entropy logic, and the kernel's TCP/UDP path, but not physical NICs, switch fabric, or wide-area routing. We retain the single-host configuration for the present revision because (i) the property under test (Theorem 2 lift onto a real transport) is independent of the physical layer, (ii) the same pattern, single-host containers + `netem`, is the standard methodology used by P2P and CRDT systems papers we are aware of for partition-heal experiments, and (iii) we do not have access to a multi-host testbed during the revision cycle. The `deploy/docker-compose.yml` stack is deliberately written so that swapping the local Docker engine for a Docker Swarm or an SSH-driven multi-host `docker compose` requires only the addition of `deploy` sections referencing remote hosts; we plan to publish multi-host runs alongside the next major release once that hardware becomes available.

IoT fleet workload (E17; Perf): Tables 7–11 characterise replay on single-key synthetic logs, which understates the policy pressure a real deployment places on the gate. E17 introduces a domain-shaped workload generator (`bench_iot_fleet`, `crates/core/src/bin/bench_iot_fleet.rs`) modelling a fleet-maintenance scenario with 50 controllers, 5 vendor issuers, 20 maintenance engineers, and a mixed operation draw (70% data writes on per-controller keys, 15% TRUST/credential events, 10% GRANT, 5% REVOKE). A single partition–heal cycle is injected mid-log: side A (vendors) and side B (one engineer chosen as the target) accumulate ops in isolation, side A issues a REVOKE for the target engineer at the start of the window, and side B continues to extend the engineer's parent chain with data writes whose HLCs strictly succeed the revoke. After heal, both side streams are merged into a single DAG, replayed deterministically, and the gate is expected to skip every side-B post-revoke write (plus any other writes whose grant epochs were closed by the random in-stream revokes). E17 reports replay time, applied/skipped counts, peak resident-set size, and the converged state digest. Because the generator is deterministic (fixed seed), the digest is a regression signature that any future model change must reproduce.

TABLE 14
IoT fleet workload (E17). **Skipped (revoked)** totals all data ops the deny-wins gate refused at replay time; the parenthesised count is the side-B partition writes from the revoked engineer alone, of which all are expected to be skipped. Peak RSS is read from `/proc/self/status` (VmHWM) at process exit; it includes generation buffers, the merged DAG, and the materialised state.

Ops	Replay (ms)	Applied	Skipped (revoked)	Peak RSS (MB)
10,000	1,078	5,957	1,413 (1,000)	22
50,000	63,519	30,663	5,767 (5,000)	116
100,000	503,062	59,841	12,769 (10,000)	230

REVOKE rate sensitivity (Q3): The reviewer (Q3, 10 April 2026) asked how E17's headline numbers shift as the fraction of operations devoted to REVOKE changes. We reran `bench_iot_fleet` at 10K ops with `--revoke-pct` $\in \{1, 5, 10, 20\}$, holding all other generator parameters fixed; the side-B partition writer always emits exactly 1,000 post-revoke writes, so any additional skipped ops at higher revoke rates come from in-stream revocations closing other engineers' grant epochs (Table 15). The trend is monotone in the expected direction: as the revoke percentage rises from 1% to 20%, applied data ops fall from 7,069 to 3,114, skipped ops climb from 460 to 3,048, and replay time actually *decreases* from 1,283 ms to 762 ms because every skipped op short-circuits the MVReg HB-aware put loop (Table 9) instead of executing it. The applied/skipped counts always sum exactly to the generated `data_ops` count, and the side-B 1,000-write floor is preserved at every revoke rate, so the sensitivity sweep does not change the qualitative finding that the gate denies every post-revoke write under the deterministic replay; it only redistributes the deny budget between in-stream revocations and the partition window.

The previous revision of this table reported zero skipped ops at every size, which the reviewer correctly flagged as suspicious. The cause was a workload-generator bug: the partition window was implemented as a bare counter, and the bench never produced post-revoke writes from the revoked engineer on the partitioned side, so there was no concurrent revocation for the gate to act on. The 10 April 2026 revision fixes this by maintaining two independent op streams during the partition window (side A keeps its own parent chain; side B replays from a snapshot taken at the moment of partition), seeding the side-B writer's HLC strictly above the side-A revoke's HLC, and reading the applied/skipped counts directly from the metrics registry rather than approximating them from `data_ops`. The applied + skipped totals now equal the generated `data_ops` count exactly, and side-B's 1,000/5,000/10,000 post-revoke writes are all denied as expected (the additional skipped ops at each row reflect the random in-stream revokes that close grant epochs for other engineers later in the workload).

TABLE 15
Q3 sensitivity sweep on the E17 IoT fleet workload (10,000 ops, otherwise default `bench_iot_fleet` configuration). `revoke_pct` sets the per-tick REVOKE draw probability; the remaining 100−`revoke_pct` is split proportionally across DATA/TRUST/GRANT. Replay time *falls* as the revoke rate climbs because skipped ops bypass MVReg merge resolution.

revoke_pct	data_ops	revoke_ops	Applied	Skipped	Replay (ms)
1%	7,529	94	7,069	460	1,283
5%	7,370	432	5,957	1,413	1,073
10%	6,966	869	4,922	2,044	1,025
20%	6,162	1,842	3,114	3,048	762

Adversarial resilience (E18, E19; T1–T4): E16 and E17 measure benign-fault behaviour: nodes go offline, recover, and merge cleanly. The reviewer (W3, 10 April 2026) asks for adversarial scenarios in which an attacker actively tries to subvert the gate. We add two experiments, both run by the dedicated harness `bench_adversarial` (`crates/core/src/bin/bench_adversarial.rs`), which collapses each scenario to an applied/skipped pair read from the metrics registry and asserts the expected outcome inside the bench so any regression fails CI.

E18 covers *credential compromise*. A subject with a valid grant emits 50 legitimate writes; the issuer then revokes the grant; the subject (assumed compromised) keeps writing 50 further data ops with HLCs strictly later than the revoke. After the merged log is replayed, the gate skips every one of the 50 post-revoke writes (50 applied, 50 skipped). E18 also includes a *forged grant* sub-scenario: a rogue issuer mints a grant for the attacker referencing a credential whose `iss` claim points at an issuer that has no `IssuerKey` on the log. `TrustView::select_key` returns `None`, the VC fails verification, no epoch opens, and the 10 data writes the attacker subsequently emits are all denied at the gate.

E19 covers *forged event injection*. (a) **Signature forgery**: a data op is signed legitimately, then its payload bytes are mutated after the signature is computed. `Op::verify` recomputes the canonical-CBOR hash over the header, observes that it no longer matches the recorded `op_id`, and the op is dropped at ingest before reaching the deny-wins gate. The bench injects one such op alongside one in-window legitimate write and confirms that exactly the

legitimate write applies. (b) **Fabricated HLC**: the attacker signs a syntactically valid op whose HLC is fabricated to lie strictly after the credential's `exp_ms`. The op ingests cleanly (signature is valid) but the gate denies it because the epoch's HLC window has elapsed. (c) **Duplicate `op_id`**: the same op is inserted twice into the merged DAG. `Dag::insert` short-circuits on a known `op_id`, so the underlying op materialises exactly once. Table 16 summarises the five outcomes.

TABLE 16
Adversarial resilience (E18, E19). All five scenarios pass; the bench panics on any deviation from the expected applied/skipped counts.

Scenario	Injected	Applied	Skipped	Outcome
E18 credential revoked	50	50	50	✓ (gate denies post-revoke)
E18 forged grant (unknown iss.)	10	0	10	✓ (no epoch opens)
E19 signature forgery	1	1	0	✓ (dropped pre-gate)
E19 HLC after exp	1	1	1	✓ (epoch expired)
E19 duplicate `op_id`	1	1	0	✓ (DAG dedup)

These experiments are deliberately small: each scenario isolates one mechanism so the verdict is unambiguous. Larger adversarial workloads (e.g., concurrent compromise of multiple credentials, racing forged grants against legitimate ones across a partition) are a natural extension and would benefit from a fuzzing harness rather than hand-written bench scenarios.

4.4 Limitations and Threats to Validity

Disconnected nodes cannot learn of revocations until they sync [33]. The vulnerability window equals partition duration; operations written by a revoked user are invalidated on reconnection, but not before. The append-only log grows monotonically, and the full log must be retained for audit. Log compaction with verifiable snapshots could bound storage but remains future work. Long-offline nodes face a proportionally large replay on reconnection; checkpoints help but do not eliminate this. Per-tag encryption protects payloads, yet operation structure, timestamps, and producer keys remain visible; full metadata privacy would require anonymous credentials, Attribute-Based Encryption (ABE), or zero-knowledge proofs [34], [35]. Revocation also cannot undo physical-world actions already taken (e.g., an actuator that has fired). Applications needing staged commit must handle this themselves.

Regarding validity, E6 covers six log sizes with five trials each, but E8 and E9 use fewer sizes and a single workload. The workstation numbers come from one Linux x86_64 machine; E15 covers edge-device replay on Raspberry Pi 4-class hardware; E16 replaces the E12 in-process transport with a real libp2p stack under network emulation; E17 adds a domain-shaped IoT-fleet workload. The workloads remain synthetic (E17 adds a domain-shaped generator, but validation on production traces is still future work), and mechanized verification of the theorems (Section 3.7) is not in scope.

5 Discussion

Partition-duration trade-off: EQUACK's deny-wins gate repairs authorization state after partition heal but cannot prevent unauthorized effects during the partition itself. The vulnerability window equals partition duration ($W(r) \leq \Delta_{\text{partition}} + \Delta_{\text{propagation}}$), a revoked user's writes are retroactively invalidated when logs merge, yet physical-world consequences (actuator commands, parameter changes) persist. The guarantee is that the final digital state $s^\star$ is always consistent with the revocation, the physical world cannot be rewound. For short disconnections (minutes to hours), operators can defer safety-critical actuations until sync confirms authorization; for day-long partitions, application-layer compensating controls (staged commit, human-in-the-loop) are needed.

Log compaction sketch: The append-only log is the source of truth for replay; without compaction it grows monotonically (E8: ≈430 B/op CBOR, ≈800 B/op on disk after RocksDB amplification, so a 500K-op deployment occupies ≈555 MB). The `store::checkpoint` module already produces verifiable state snapshots used by E6's incremental replay path; extending it to a compaction primitive requires only tying the snapshot to a DAG *watermark*: a frontier of op_ids such that every event causally before the frontier has been incorporated into the snapshot's state digest. Once a watermark is anchored (signed by the same stakeholders that sign the genesis, and pinned in the audit chain), events strictly before it can be pruned from local storage; new replicas bootstrap by fetching the snapshot, the watermark anchor, and the suffix DAG. Convergence (Theorem 2) is preserved because the snapshot replaces a deterministic prefix replay with its byte-identical result, and audit tamper-evidence (Theorem 4) is preserved because the audit-chain head pins the watermark. As an order-of-magnitude estimate, a 500K-op deployment compacted to a ∼50K-op suffix plus a ∼200 B snapshot would shrink from ≈555 MB to ≈60 MB (an ≈9× reduction), with the per-replica replay window proportionally reduced. Compaction is not implemented in the prototype; the open design issues are watermark-anchor governance (which stakeholders sign, how often) and how to handle long-disconnected nodes whose own un-merged events predate a published watermark.

Hold-and-confirm pattern for safety-critical actuation: For deployments where physical-world actions are irreversible (e.g., actuator commands, safety-parameter changes applied to running equipment), we recommend a two-phase pattern: (1) the controller records the requested change in the event log and applies it to a *pending* state visible to the local operator, but (2) defers commitment to the *operational* state until the next sync cycle confirms that the underlying credential remains valid in the converged replay and no intervening REVOKE has closed the epoch. This hold-and-confirm pattern is not part of EQUACK's model; it is an application-layer policy that uses EQUACK's replay verdicts as input. The trade-off is availability: pending changes are visible but not yet authoritative, which may be acceptable for scheduled maintenance windows but not for emergency operations. A variant for short disconnections lowers the latency floor by pairing hold-and-confirm with a local "freshness budget" (a maximum tolerable age of the status evidence), beyond which the controller degrades to a safe fallback mode instead of acting optimistically.

Scalability: Memory grows at ≈1.7 KB/op (Table 11), so a 256 MB edge device holds roughly 150K operations, enough for weeks of maintenance logs, but not high-frequency telemetry. Throughput drops from 45K ops/s on linear chains to 7K ops/s with eight concurrent writers (Table 8); at ≈430 bytes/op in CBOR, a million-operation log occupies ∼430 MB before write amplification. Without log compaction (deferred to future work), long-lived deployments will need archival. These constraints position EQUACK for authorization-rate workloads on devices with modest storage rather than high-throughput data-plane enforcement.

Deny-wins conservatism: The deny-wins semantics are deliberately conservative; concurrent GRANT and REVOKE resolves in favor of revocation, appropriate where over-granting is riskier than over-denying. An erroneous revocation retroactively voids subsequent work; recovery requires a new GRANT to open a fresh epoch (Section 3.6), which may not be timely during disconnection. Making the combining algorithm configurable per scope is a natural extension, though it would complicate the convergence argument.

Clock skew: EQUACK relies on the physical component of the HLC for credential validity checks. In deployments without trusted time sources (e.g., GPS, Network Time Protocol (NTP)), clock skew can cause a credential to appear valid on one replica and expired on another for events near validity boundaries. The formal property (Theorem 1) is unaffected: determinism depends on the HLC values recorded in the log, not on their accuracy relative to real time. However, the *intended* authorization semantics—that a credential should not be usable outside its validity interval—degrade proportionally to clock skew. Deployments should either provision trusted time sources or set validity margins exceeding the expected skew bound ϵ (Section 2.2, item 7). Operators facing drift-prone edge devices can also reduce boundary ambiguity by issuing credentials with coarse-grained (e.g., hour- or day-level) validity windows relative to the expected skew.

In-band trust: Carrying issuer keys and status-list updates as replicated log events eliminates online PKI lookups, enabling offline VC verification. The cost is that TrustView grows with each key registration and status-list chunk; deployments with frequent key rotations across many issuers would see meaningful log growth from trust material alone. The pinned genesis checkpoint anchors the trust chain; its governance (who signs, how keys are distributed, how organizations join or leave) is an open design problem.

Genesis attack surface: The genesis checkpoint is the trust root of every EQUACK deployment, so its compromise invalidates every downstream guarantee. If an attacker substitutes a genesis carrying their own issuer keys before any victim replica pins it, that replica's TrustView accepts attacker-issued credentials as legitimate; Theorems 1–3 still hold over the resulting (corrupted) DAG, but the deny-wins gate is now defending the wrong perimeter. The relevant attack surface is therefore the out-of-band distribution channel for the genesis digest, not the replay machinery itself. We recommend three layered mitigations: (1) *multi-party cross-signing*, in which each founding stakeholder independently signs the genesis and replicas accept it only when at least k-of-n signatures verify, raising the bar from one compromised key to k; (2) *public anchoring*, in which the genesis digest is pinned in a tamper-evident transparency log or a public-blockchain anchor, used not for consensus but for non-equivocation, so that any divergence between the digest a replica saw and the digest the rest of the world saw is publicly detectable; and (3) *hardware-backed key custody* for the genesis-signing keys themselves (HSMs or smartcards with audited firmware), reducing the chance of silent key extraction. The prototype's single-admin genesis is a development convenience and should be read as the minimum-viable bootstrap, not a production recommendation. *Split-brain genesis*, in which two disjoint groups of stakeholders each produce a self-consistent but incompatible genesis, is the same problem in another form: the result is two non-interoperable trust domains whose existence cannot be detected from inside either domain. Detection requires out-of-band comparison of pinned digests (either via the public anchor in mitigation (2) or by direct human governance), and resolution requires human action: the protocol cannot automatically reconcile incompatible trust roots, because doing so would require trusting one side over the other — exactly the decision the genesis was supposed to settle in the first place.

Online-mode authorization latency: EQUACK is engineered for the offline case, but the same gate runs online. Isolating a single Gate decision (policy check + epoch lookup, no signature verification, no DAG insertion, no CRDT merge) in the `bench_gate` microbenchmark over 10,000 iterations after a 1,024-iteration warm-up, we measure ≈96 ns mean per call, with p50/p95/p99 of 95/102/103 ns and a worst-case observed value of 5.9 μs on the bench host (one IssuerKey + Credential + Grant in the index, single granted subject, scope $\{\text{hv}, \text{mech}\}$). At $\sim 10^{-7}$ s per decision, the gate adds roughly an order of magnitude over a primary-key RBAC table lookup but lands in the same regime as Cedar's reported evaluation latency on policies of comparable size. Deployments with reliable connectivity, a single trust authority, and no requirement for retroactive revocation safety should still prefer a centralised policy engine; EQUACK's value lies in workloads where the offline guarantees (R1, R3, R5, Section 2.3) are needed.

Comparison scope: Tables 1 and 2 evaluate EQUACK against paradigm categories rather than specific systems, because we are not aware of an existing system that combines CRDT replication with in-band VCs and a deny-wins gate. The key distinction from application-level access control is enforcement timing; application-level checks execute at ingest and cannot retroactively invalidate operations accepted under stale evidence; EQUACK's replay-time gate can (R3). The closest system is p2panda, whose retroactive replay of application data is optional and application-implemented, whereas EQUACK's deny-wins gate is mandatory during deterministic replay. A direct empirical comparison is infeasible as p2panda provides no peer-reviewed formal model or published benchmarks.

Scoped replication: The prototype assumes *full replication*: every replica eventually receives every event in the DAG. For deployments where this is impractical (e.g., 50 controllers × 100K ops/each, where no single replica needs the union), *scoped replication* is a natural extension. The

DAG structure already supports it: a replica can maintain a subgraph rooted at the resource tags it manages and gossip only events whose payload tags match a configured subscription filter. Scoped replication does not affect Theorem 2 (Convergence) for the subscribed subgraph, because the proof is over the subset of events a given replica actually ingests; what it requires is careful handling of cross-scope REVOKE events (a credential revocation that closes epochs spanning more than one tag must reach every replica subscribed to any of those tags) and of TRUST events (issuer keys and status-list chunks have to flow to every replica that might verify a credential touching the affected scope). Designing the subscription topology so these cross-cutting events propagate without re-introducing full replication is left for future work.

Cedar integration roadmap: The current policy engine is a ∼1,000-LOC purpose-built evaluator (`core::policy`) covering the Cedar subset enumerated alongside Listing 1. It exists because the prototype predates the stabilisation of the upstream Cedar Rust SDK (`cedar-policy` crate). Substituting the upstream SDK as a drop-in replacement for the evaluator is feasible: EQUACK's gate already calls the policy engine through a single internal interface (subject + action + resource tags + epoch index + position → allow/-deny), and the upstream SDK provides exactly that API plus full type checking, schema validation, partial evaluation, and the verified properties cited by the Cedar team. The migration is on the roadmap but is not prioritised over the mechanised verification work for the replay–gate composition (Theorems 1–3); the latter is a paradigm-level concern that the upstream SDK cannot answer for us, whereas the former is a purely engineering task that can be done at any time without affecting the formal claims.

Prototype scope and Cedar properties: Cedar's verified properties (decidability, totality, soundness) hold for the converged state $s^\star$ but not for transient local decisions during partitions, an inherent limitation of any eventually consistent system embedding a single-context policy engine. The prototype implements the full conceptual model with simplifications; VC validation omits JSON-LD and selective disclosure; key management uses X25519 without a full ceremony protocol; Cedar's full type system is not implemented; and libp2p gossip uses basic flooding. The evaluation uses synthetic workloads on a single Linux x86_64 machine; E12 simulates multi-node sync in-process; cross-platform determinism is confirmed between x86_64 and aarch64 (E15); and the confidentiality layer is experimental. The formal claims (Theorems 1–5) are argued but not mechanically verified; mechanized verification of the replay-gate composition is a priority for future work.

6 Conclusion

Offline multi-stakeholder systems need authorization decisions that remain available during disconnection and converge deterministically after partition healing. The ECAC paradigm [8] establishes that such convergence is formally achievable, but leaves open how to realize it with concrete credentials, fine-grained policies, and in-band trust. EQUACK closes this gap by recording data operations, credential changes, and trust evidence in a single signed causal event log and reconstructing authorized state through deterministic deny-wins replay.

The prototype evaluation supports this within a single-machine, synthetic-workload setup. One hundred valid orderings of a 10 000-operation log all converged to the same state digest; a revocation workload suppressed every post-revocation write after reconciliation; multi-issuer conflicts resolved deterministically; and audit tampering was detected. Replay throughput reached 45 000 operations/s on linear chains, and checkpoint restore cut replay cost by 8–11×.

A credential revocation issued by any authorized issuer eventually takes effect at every replica, regardless of partition topology or message ordering. The in-band trust model eliminates the dependency on always-online PKI infrastructure, and the tamper-evident audit trail allows independent parties to verify that authorization decisions were computed correctly from the same evidence. Together, these properties enable offline authorization in settings, industrial IoT, federated data sharing, disaster response, where, to the best of our knowledge, no prior system surveyed in Sections 2.4–2.5 simultaneously provides deterministic convergence, retroactive revocation safety, and fine-grained policy expressiveness.

The workloads are synthetic, the multi-node topology is simulated in-process, and the confidentiality layer is experimental. The model trades real-time revocation for partition availability, it guarantees the converged *digital* state, not reversal of physical-world actions. Multi-machine evaluation under realistic network partitions and mechanized verification of the replay-gate composition are the priority next steps.

References

[1] I. Mistry, S. Tanwar, S. Tyagi, and N. Kumar, "Blockchain for 5g-enabled iot for industrial automation: A systematic review, solutions, and challenges," *Mechanical Systems and Signal Processing*, vol. 135, p. 106382, 2020.

[2] H. Liu, D. Han, and D. Li, "Fabric-iot: A blockchain-based access control system in iot," *IEEE Access*, vol. 8, pp. 18 207–18 218, 2020.

[3] A. Hasselgren, K. Kralevska, D. Gligoroski, S. A. Pedersen, and A. Faxvaag, "Blockchain in healthcare and health sciences—a scoping review," *International Journal of Medical Informatics*, vol. 134, p. 104040, 2020.

[4] K. Gai, Y. She, L. Zhu, K.-K. R. Choo, and Z. Wan, "A blockchain-based access control scheme for zero trust cross-organizational data sharing," *ACM Trans. Internet Technol.*, vol. 23, no. 3, Aug. 2023.

[5] Z. Du, Y. Li, Y. Fu, and X. Zheng, "Blockchain-based access control architecture for multi-domain environments," *Pervasive and Mobile Computing*, vol. 98, p. 101878, 2024.

[6] D. Han, Y. Zhu, D. Li, W. Liang, A. Souri, and K.-C. Li, "A blockchain-based auditable access control system for private data in service-centric iot environments," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 5, pp. 3530–3540, 2022.

[7] T. Hu, S. Yang, Y. Wang, G. Li, Y. Wang, G. Wang, and M. Yin, "N-accesses: A blockchain-based access control framework for secure iot data management," *Sensors*, vol. 23, no. 20, 2023.

[8] F. Jacob and H. Hartenstein, "To the best of knowledge and belief: On eventually consistent access control," in *Proceedings of the 15th ACM Conference on Data and Application Security and Privacy*, 2024.

[9] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*. ACM, 2019, pp. 154–178.

[10] S. Rouhani, R. Belchior, R. S. Cruz, and R. Deters, "Distributed attribute-based access control system using permissioned blockchain," *World Wide Web*, vol. 24, no. 5, pp. 1617–1644, 2021.

[11] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, ser. Lecture Notes in Computer Science, vol. 6976. Springer, 2011, pp. 386–400.

[12] V. B. F. Gomes, M. Kleppmann, D. Mulligan, and A. R. Beresford, "Verifying strong eventual consistency in distributed systems," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, 2017.

[13] G. Yu, X. Zha, X. Wang, W. Ni, K. Yu, P. Yu, J. A. Zhang, R. P. Liu, and Y. J. Guo, "Enabling attribute revocation for fine-grained access control in blockchain-iot systems," *IEEE Transactions on Engineering Management*, vol. 67, no. 4, pp. 1213–1230, 2020.

[14] D. Han, N. Pan, and K.-C. Li, "A traceable and revocable ciphertext-policy attribute-based encryption scheme based on privacy protection," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 1, pp. 316–327, 2022.

[15] M. Xiao, Q. Huang, Y. Miao, S. Li, and W. Susilo, "Blockchain based multi-authority fine-grained access control system with flexible revocation," *IEEE Transactions on Services Computing*, vol. 15, no. 6, pp. 3143–3155, 2022.

[16] Y. Miao, R. H. Deng, X. Liu, K.-K. R. Choo, H. Wu, and H. Li, "Multi-authority attribute-based keyword search over encrypted cloud data," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 4, pp. 1667–1680, 2021.

[17] K. Yue, Y. Zhang, Y. Chen, Y. Li, L. Zhao, C. Rong, and L. Chen, "A survey of decentralizing applications via blockchain: The 5g and beyond perspective," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2191–2217, 2021.

[18] D. Di Francesco Maesa, P. Mori, and L. Ricci, "A blockchain based approach for the definition of auditable access control systems," *Computers & Security*, vol. 84, pp. 93–119, 2019.

[19] H. Rathore, A. Mohamed, and M. Guizani, "A survey of blockchain enabled cyber-physical systems," *Sensors*, vol. 20, no. 1, 2020.

[20] M. M. Merlec, M. M. Islam, Y. K. Lee, and H. P. In, "A consortium blockchain-based secure and trusted electronic portfolio management scheme," *Sensors*, vol. 22, no. 3, 2022.

[21] A. Birgisson, J. G. Politz, U. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud," in *Proceedings of the 2014 Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2014.

[22] B. Zelenka, P. Krüger *et al.*, "UCAN specification v0.10.0," https://github.com/ucan-wg/spec, 2022, accessed: 2025.

[23] M. P. Andersen, S. Kumar, M. AbdelBaky, G. Fierro, J. Kolb, H.-S. Kim, D. E. Culler, and R. A. Popa, "WAVE: A decentralized authorization framework with transitive delegation," in *Proceedings of the 28th USENIX Security Symposium*. USENIX Association, 2019, pp. 1375–1392.

[24] G. Couprie *et al.*, "Biscuit: Bearer tokens with offline attenuation and decentralized verification," https://www.biscuitsec.org/, 2023, accessed: 2025.

[25] A. Meyer and S. Gwilym, "Willow protocol specification," https://willowprotocol.org/, 2024, includes Meadowcap capability system. Accessed: 2025.

[26] R. Pang, R. Caceres, M. Burrows, Z. Chen, P. Dave, N. Gerber, A. Giber, A. Narayan, and F. Boemer, "Zanzibar: Google's consistent, global authorization system," in *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC)*. USENIX Association, 2019, pp. 33–46.

[27] J. W. Cutler, C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, E. Ioannidis, J. Kastner, A. Mamat, D. McAdams, M. McCutchen, N. Rungta, E. Torlak, and A. Wells, "Cedar: A new language for expressive, fast, analyzable, and secure authorization," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, 2024.

[28] The Matrix.org Foundation, "Matrix specification: State resolution v2," https://spec.matrix.org/v1.8/rooms/v2/, 2023, accessed: 2025.

[29] adz, S. Andreae, and glyph, "p2panda: A protocol for local-first applications," https://p2panda.org/, 2024, v0.5.2, March 2025. Rust toolkit for P2P local-first apps. Accessed: 2025.

[30] adz, "Notes on building a convergent, offline-first access control CRDT," https://p2panda.org/2025/08/27/notes-convergent-access-control-crdt.html, 2025, describes strong-removal resolver and convergence argument. Accessed: 2025.

[31] ——, "Local-first group- and message encryption in p2panda," https://p2panda.org/2025/02/24/group-encryption.html, 2025, dCGKA-based encryption with key rotation. Accessed: 2025.

[32] World Wide Web Consortium, "Verifiable credentials data model v2.0," https://www.w3.org/TR/vc-data-model-2.0/, 2024, w3C Recommendation. Accessed: 2025.

[33] T. Bui and T. Aura, "Application of public ledgers to revocation in distributed access control," in *Information and Communications Security*, D. Naccache, S. Xu, S. Qing, P. Samarati, G. Blanc, R. Lu, Z. Zhang, and A. Meddahi, Eds. Cham: Springer International Publishing, 2018, pp. 781–792.

[34] J. Bernal Bernabe, J. L. Canovas, J. L. Hernandez-Ramos, R. Torres Moreno, and A. Skarmeta, "Privacy-preserving solutions for blockchain: Review and challenges," *IEEE Access*, vol. 7, pp. 164 908–164 940, 2019.

[35] Y. Zhang, R. H. Deng, S. Xu, J. Sun, Q. Li, and D. Zheng, "Attribute-based encryption for cloud computing access control: A survey," *ACM Computing Surveys (CSUR)*, vol. 53, no. 4, pp. 1–41, 2020.

Aditya Sissodiya received his B.Sc. (Hons.) degree in Computer Science from Doon University, and his M.Sc. degree in Computer Science from Christ University, India. He is pursuing a Ph.D. in Cyber-Physical Systems at Luleå University of Technology, Sweden. Before beginning his doctoral studies, he worked as a software engineer for over 3 years in industry, contributing to supply chain optimization and healthcare technologies. His research interests lie in access control and secure collaboration mechanisms, with a recent focus on technologies supporting the circular economy.

Johan Kristiansson received his Ph.D. in multimedia technology from Luleå University of Technology, where he is currently employed as a Senior Lecturer in Cyber-Physical Systems. He has extensive industry experience, including over a decade at Ericsson, where he worked as a Principal Researcher specialising in container orchestration. As an early adopter of Docker, he explored microservices architectures and container security, and developed an in-house orchestration platform at Ericsson in 2014. He later served as CTO at RockSigma AB, where he built a Kubernetes-based seismic processing engine. He is also the lead developer of ColonyOS, a meta-OS designed to create computing continuums. Dr. Kristiansson has authored over 25 scientific publications and holds 23 patents in distributed computing, cloud computing, and container orchestration.

Eric Chiquito received his PhD in Cyber-Physical Systems from Luleå University of Technology (LTU) in October 2024 and currently serves as a Postdoctoral Researcher in the same research area. His research focuses on distributed systems, sustainability-oriented digitalization, and the design of digital architectures for environmental and regulatory reporting, including Digital Product Passports (DPP). Dr. Chiquito has participated in multiple European and Swedish research initiatives where academic research is closely integrated with industrial partners.

Ulf Bodin is a professor at the Luleå University of Technology, where he is conducting research on the industrial IoT, distributed system of systems, computer communications, distributed ledgers and applied machine learning. He has a PhD in computer networking from Luleå University of Technology and more than 15 years of experience in academia and the software industry, including standardization in ETSI and other organizations.